

Trabajo Final - Visión Artificial

Inteligencia Artificial I 2025

Alumno: CORAZZA, Luis Agustín - 12909

Fecha de examen: 12/02/2025

Profesores:

- **Dra. Ing. Selva Rivera**
- **Ing. Juan García**

Contenido

1.	Resumen	4
2.	Introducción	4
2.1.	Etapa 1	4
2.2.	Etapa 2	5
2.3.	Etapa 3	5
3.	Estado del arte.....	5
3.1.	Reconocimiento de imágenes.....	5
3.1.1.	K-Means.....	5
3.1.2.	KNN.....	7
3.2.	Planificación Automática: STRIPS	8
3.3.	Algoritmos de búsqueda.....	8
3.3.1.	Algoritmo de Búsqueda Primero en Anchura (BPA).....	8
3.3.2.	Algoritmo A*.....	9
3.3.3.	Algoritmo A* modificado.....	9
3.4.	Generación automática de laberintos: Algoritmo aleatorio de Prim	10
4.	Especificación del agente	10
4.1.	Propiedades del entorno	11
4.2.	Estructura del agente.....	12
5.	Diseño del agente	13
5.1.	Etapa 1 - Identificación de objetos mediante visión artificial.....	13
5.1.1.	Procesamiento de imágenes	14
5.1.2.	Detección de contornos	15
5.1.3.	Extracción de características	16
5.1.4.	Selección de características.....	17
5.1.5.	Procesamiento de las características	20
5.1.6.	Algoritmo kNN	20
5.1.7.	Algoritmo K-Means.....	22
5.2.	Etapa 2 - Planificación y re-apilamiento de las cajas	24
5.3.	Etapa 3 - Transporte de las cajas	26
6.	Ejemplo de aplicación.....	27
7.	Resultados	29
8.	Conclusiones.....	30
9.	Referencias bibliográficas.....	30

10.	Anexo: Código desarrollado	32
10.1.	Clase base para los algoritmos de identificación de objetos.....	32
10.2.	Algoritmo KNN	34
10.3.	Algoritmo KMeans	35
10.4.	Clase base para los algoritmos de búsqueda.....	37
10.5.	Algoritmo A*	38
10.6.	Algoritmo A* modificado	39
10.7.	Algoritmo BPA.....	40
10.8.	Clase correspondiente al Laberinto	41
10.9.	Clase al controlador de la interfaz	42
10.10.	Clase correspondiente a la ventana del laberinto.....	45
10.11.	Clase correspondiente a la ventana del reordenamiento de cajas	47
10.12.	Clase correspondiente a la ventana de identificación de objetos	50

1. Resumen

En este trabajo se diseñó un sistema de visión artificial basado en los algoritmos K-Means y K-NN para identificar el contenido de cuatro cajas apiladas con tornillos, tuercas, clavos y arandelas. Para entrenar los algoritmos de identificación, se recolectaron imágenes individuales de cada objeto y se construyó una base de datos. Posteriormente, ambos algoritmos fueron implementados y evaluados en términos de precisión y eficiencia.

Además, en una segunda instancia, se empleó el lenguaje STRIPS para planificar la reorganización de las cajas en una nueva configuración. Mediante la herramienta Fast Downward, se determinó la secuencia óptima de acciones que minimizan el costo del plan de reordenamiento.

Por otro lado, se implementaron distintos algoritmos de búsqueda para la planificación de rutas, con especial énfasis en A*, utilizando la heurística de Distancia de Manhattan. Este enfoque permitió optimizar el desplazamiento del robot en un laberinto generado aleatoriamente, asegurando una trayectoria eficiente desde un punto A hasta un punto B. Adicionalmente, se exploró el desarrollo de un algoritmo para la generación automática de laberintos.

Los resultados obtenidos demuestran el potencial de los algoritmos desarrollados para la clasificación de objetos, la planificación de acciones y la búsqueda optimizada de rutas, contribuyendo a la mejora de la autonomía y eficiencia del sistema robótico.

2. Introducción

La Inteligencia Artificial (IA) es el campo de estudio de los agentes que reciben percepciones del entorno y llevan a cabo determinadas acciones. Uno de los campos de estudio de la IA es la Visión Artificial que permite a las máquinas interpretar y analizar imágenes o videos para extraer información relevante.

La visión artificial utiliza algoritmos de procesamiento de imágenes y aprendizaje automático para identificar patrones, reconocer objetos y tomar decisiones basadas en datos visuales. Su objetivo es replicar y mejorar la capacidad humana de percepción visual en aplicaciones como el reconocimiento facial, la detección de defectos en manufactura y la conducción autónoma.

En paralelo, el desarrollo de algoritmos de búsqueda y planificación ha sido fundamental para la automatización de tareas complejas. Inspirados en los procesos cognitivos humanos, estos algoritmos permiten a las máquinas encontrar soluciones óptimas a problemas específicos, desde la navegación de robots hasta la organización de tareas en entornos dinámicos.

Este trabajo explora algunos de los algoritmos más famosos de cada campo de estudio en una aplicación amplia que podría representar una situación real. Abarcaremos desde la identificación y clasificación de objetos, hasta la planificación y búsqueda de rutas óptimas en un entorno desconocido.

Para simplificar y ordenar el desarrollo del proyecto, se dividió al trabajo en tres etapas principales:

2.1. Etapa 1

Se requiere identificar el contenido de 4 cajas diferentes, apiladas una sobre otra, de manera secuencial mediante el reconocimiento a través de una imagen tomada en ese momento. Diferentes técnicas de procesamiento de imágenes son aplicadas a la imagen original para extraer características

útiles que permitan reconocer el contenido. Luego, dichas características serán utilizadas como entrada de dos algoritmos de Machine Learning: **KNN** y **K-Means**.

2.2. Etapa 2

En un principio, las cajas están apiladas en un orden cualquiera. Sin embargo, luego de la detección y reconocimiento de su contenido, estas deben ser apiladas en un determinado orden establecido al momento. En este sentido, se utilizará **STRIPS**, un lenguaje de descripción de problemas de planificación, utilizado para **representar** problemas de planificación en términos de estados, acciones y objetivos. Luego, un algoritmo de planificación determinado encontrará la secuencia de pasos óptima.

2.3. Etapa 3

Finalmente, las cajas luego de ser reordenadas, deberán ser transportadas hasta un punto final determinado. Para ello se utilizarán algoritmos de búsqueda (principalmente el **algoritmo A***) para hallar el camino óptimo entre el punto de inicio y el punto final. Los algoritmos utilizados deberán ser capaces de sobrepasar los diferentes obstáculos presentes en el entorno.

3. Estado del arte

En esta sección se describe la información recopilada sobre los diferentes algoritmos y tipos de agentes empleados en el desarrollo del trabajo.

3.1. Reconocimiento de imágenes

En el presente trabajo, los algoritmos empleados para realizar el reconocimiento de imágenes son K-Means y KNN. A continuación, se describen dichos algoritmos.

3.1.1. K-Means

K-Means es un algoritmo de aprendizaje no supervisado empleado para la **agrupación de datos**. Los datos de entrenamiento utilizados por este algoritmo no están etiquetados, lo que quiere decir que el algoritmo no posee ningún tipo de información previa acerca de la estructura de clasificación del conjunto de datos.

El principio de funcionamiento se basa en un **agrupamiento iterativo basado en centroides**, que divide un conjunto de datos en diferentes grupos o clusters en función de la distancia euclidiana entre cada punto y los centroides. Los puntos de datos se agrupan dentro de la misma categoría que su centroide más cercano. El centroide es la media de todos los puntos dentro del cluster.

El principal hiperparámetro a elegir es k , el número de clusters en el que se dividirán el conjunto de datos. Un valor de k más alto, implica una mayor cantidad de clusters de menor tamaño. Existen diversas técnicas para determinar el valor óptimo de k , como el **método del codo** (Ilustración 1).

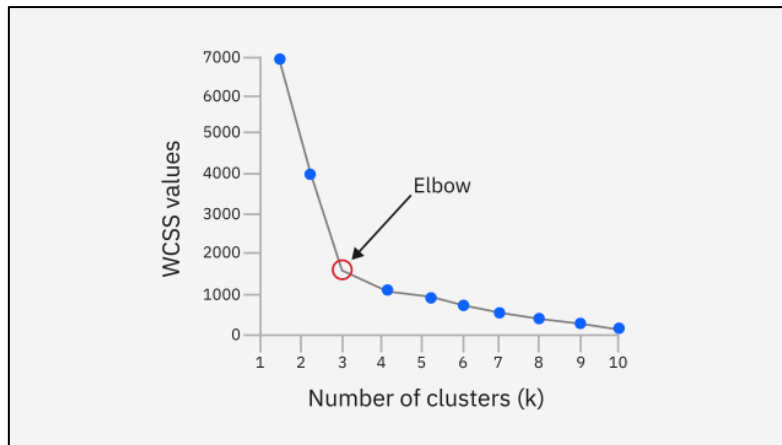


Ilustración 1. Método del codo. Fuente: IBM.

El algoritmo KMeans requiere algunos pasos, los cuales se listan a continuación:

1. **Inicialización de centroides:** El primer paso es inicializar k centroides donde k es igual al número de clústeres elegidos para un conjunto de datos específico.
2. **Asignación de centroides:** El siguiente paso incluye un proceso iterativo de dos etapas. Primero, se asigna cada punto de datos a su centroide más cercano en función de la distancia (generalmente euclidiana). Luego, se calcula la media de todos los puntos de cada conglomerado y reasigna el centroide. Este proceso se repite hasta que las posiciones de los centroides hayan alcanzado la convergencia o se haya alcanzado el número máximo de iteraciones.

Este algoritmo es sensible a las condiciones iniciales y a la presencia de valores atípicos. Por esta razón, es importante optimizar la inicialización del centroide y el número de clústeres k , para lograr los clústeres más significativos. Hay varias formas de evaluar y optimizar los componentes de agrupación del algoritmo mediante el uso de métricas de evaluación y métodos de muestreo de centroide inicial.

3.1.1.1. Métricas de evaluación de clusters

Los clusters de calidad contienen al menos dos propiedades:

1. Todos los puntos de datos dentro de un clúster deben ser similares.
2. Los clústeres deben ser distintos entre sí.

Estas propiedades se logran minimizando la distancia dentro del clúster y maximizando la distancia entre clústeres de todos los puntos de datos en un conjunto de datos. En otras palabras, cuanto más compacto y aislado esté un clúster de otros clústeres, mejor.

Entre las métricas de evaluación se destaca la **inercia o WCSS (Within-Cluster Squared Sum)**. La inercia se calcula midiendo la distancia entre un punto de datos y su centroide, elevando la distancia al cuadrado y sumando esos cuadrados para cada punto de datos del cluster. Cuanto menor sea la suma, mejor, porque significa que los puntos de datos dentro del clúster son compactos o más similares.

3.1.1.2. Inicialización de los centroides del cluster

El algoritmo K-Means es **no determinista** debido a su paso de inicialización aleatoria. Este método implica que, si el algoritmo se realiza dos veces en datos idénticos, las asignaciones del clúster pueden

diferir. Para lograr resultados óptimos de clustering, los centroides iniciales deben ser seleccionados adecuadamente.

Un método de inicialización de centroides bastante popular es **K-Means++** que optimiza la selección de los centroides iniciales. El primer paso de este algoritmo es elegir el primer centroide al azar de entre los puntos del conjunto de datos. Para cada punto restante, se calcula la distancia al centroide más cercano. Luego, se selecciona el siguiente centroide con **probabilidad proporcional al cuadrado de su distancia** al centroide más cercano. Esto significa que los puntos más alejados tienen más probabilidad de ser elegidos. Estos dos últimos pasos se repiten hasta seleccionar k centroides en total.

De esta forma, el proceso de inicialización K-Means++ mejora la estabilidad de los clusters, evitando inicializaciones deficientes. Este proceso tiene una complejidad de $O(kN)$ (en comparación con $O(N)$ de k-means clásico). Sin embargo, esta inversión de tiempo inicial suele valer la pena porque el algoritmo en general converge más rápido y las soluciones globales suelen ser mejores.

3.1.2. KNN

El algoritmo KNN (K-Nearest Neighbours) es un algoritmo supervisado de clasificación que emplea la proximidad en base a una métrica de distancia (generalmente euclidiana) para realizar predicciones acerca de la agrupación de un punto de datos individual frente a un conjunto de datos.

Este algoritmo puede ser utilizado tanto como para tareas de clasificación como para regresión. Para los problemas de clasificación, la predicción se realiza sobre la base de un voto mayoritario.

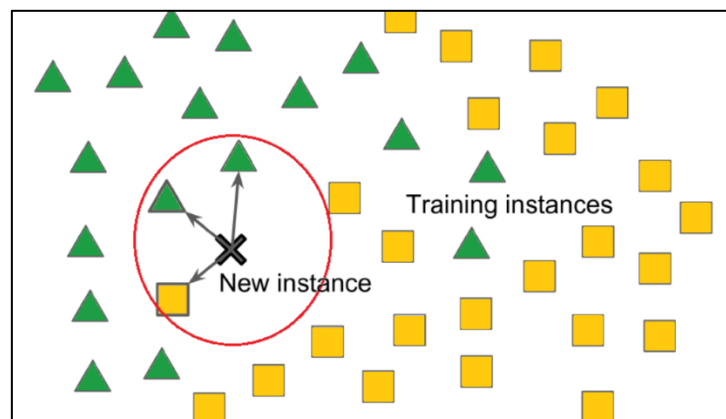


Ilustración 2. Diagrama KNN.

Cabe destacar que el algoritmo KNN forma parte de la familia de algoritmos de “aprendizaje perezoso”, lo que significa que solo almacena un conjunto de datos de entrenamiento en lugar de pasar por una etapa de entrenamiento. Esto también significa que todo el cálculo ocurre cuando se realiza una clasificación o predicción.

El valor de k en el algoritmo KNN define cuántos vecinos se verificarán para determinar la clasificación de un punto de consulta específico. Por ejemplo, si $k=1$, la instancia se asignará a la misma clase que su vecino más cercano. La elección de k dependerá en gran medida de los datos de entrada. En general, se recomienda tener un número impar para k a fin de evitar empates en la clasificación, a su vez, el valor de k debería ser siempre mayor al número de clases a predecir. Diferentes técnicas como la validación cruzada (**cross-validation**) suelen ser empleadas en este proceso de determinación de k .

3.2. Planificación Automática: STRIPS

La planificación automática es un área clave en inteligencia artificial (IA) que busca generar secuencias de acciones para alcanzar un objetivo a partir de un estado inicial. Uno de los primeros modelos formales de planificación fue **STRIPS (Stanford Research Institute Problem Solver)**, desarrollado por Fikes y Nilsson (1971), que introdujo una representación estructurada del problema basada en **estados, acciones y objetivos**. STRIPS se convirtió en el estándar para describir problemas de planificación y ha influenciado el desarrollo de numerosos enfoques posteriores.

STRIPS modela problemas de planificación a través de una representación lógica en la que:

- Los **estados** se describen como conjuntos de proposiciones o predicados.
- Las **acciones** tienen **precondiciones** (requisitos para ejecutarlas) y **efectos** (cambios en el estado tras su aplicación).
- Los **objetivos** son condiciones que el sistema debe alcanzar.

Este enfoque permitió representar problemas de manera compacta y facilitó la automatización de la planificación, aunque presentaba ciertas limitaciones, como la falta de modelado de incertidumbre o restricciones temporales.

A partir de STRIPS, se han desarrollado diversas extensiones para abordar sus limitaciones. Entre ellas, se destacan, por ejemplo: ADL, PDDL y Fast-Downward.

3.3. Algoritmos de búsqueda

Un algoritmo de búsqueda es un conjunto sistemático de procedimientos diseñado para encontrar un elemento o un conjunto de elementos dentro de una estructura de datos como puede ser un grafo o un árbol, a la cual se la denomina **espacio de estados**. Existen diversos algoritmos de búsqueda, cada uno adecuado para diferentes situaciones (tamaño del conjunto de datos a explorar, requerimientos del problema, entre otros).

Los algoritmos de búsqueda pueden clasificarse en **informados** o **no informados**. Cuando el sistema agente posee algún tipo de información del medio se habla de técnicas de búsquedas informadas; sin embargo, si carece de conocimiento alguno, llevan el nombre de algoritmos de búsqueda no informadas. Los algoritmos de búsqueda no informados suelen ser más lentos y costosos en memoria. Entre los algoritmos de búsqueda más populares se encuentra el algoritmo A* (A-estrella) y el algoritmo búsqueda primero en anchura.

Numerosos estudios tratan sobre la utilización de algoritmos de búsqueda para la búsqueda de caminos óptimos en el contexto de un laberinto 2D. El artículo realizado por Ade Candra [3] analiza la utilización directa del algoritmo A* en este contexto, y muestra su lógica de funcionamiento en diagrama de bloques y su implementación en pseudocódigo, lo cual facilita su desarrollo en un entorno real de programación como Python.

3.3.1. Algoritmo de Búsqueda Primero en Anchura (BPA)

El algoritmo de búsqueda primero en anchura (BPA), es un algoritmo de búsqueda no informado cuyo principio de funcionamiento consiste en ir explorando los nodos a medida que parten del origen. La búsqueda comienza por el nodo raíz y visita sucesivamente los nodos más cercanos a la raíz hasta que el nodo destino es alcanzado, o hasta que no queda ningún nodo por explorar, lo que ocurra primero. La función de evaluación de este algoritmo está dada por:

$$f(n) = g(n)$$

donde n representa el nodo a evaluar y $g(n)$ representa el costo total de la ruta desde el nodo de inicio hasta n , en este caso, **$g(n)$ denota el número de nodos que hace falta visitar hasta llegar a n a partir del nodo de inicio.**

Durante el proceso de exploración del entorno, el algoritmo comienza analizando el nodo raíz y añade a una cola de prioridad a todos los nodos vecinos. En la cola, cada nodo se asocia con el valor dado por la función de evaluación, donde se les da mayor prioridad a los nodos de menor valor. De esta forma, puesto que la función de evaluación implica en definitiva la distancia del nodo n al nodo de inicio, **BPA explora primero el nodo de la cola que se encuentre más cercano al nodo raíz.**

Cada vez que un nuevo nodo es visitado, se comprueba si es el nodo objetivo. En caso de que no lo sea, se añaden sus nodos vecinos a la cola de prioridad y se reordena la cola en función de los valores de $f(n)$. El proceso se repite hasta que el nodo analizado corresponde al nodo destino, o hasta que no quedan nodos por analizar. El camino obtenido contiene el mínimo número de nodos, es decir, es el camino más corto medido en número de nodos.

La complejidad de tiempo del algoritmo es $O(|N|+|A|)$ donde $|N|$ es el número de nodos transitables del entorno y $|A|$ es el número de aristas de cada nodo. El razonamiento es porque en el peor caso, cada vértice y cada arista será visitado por el algoritmo.

Una vez el algoritmo halla un camino, el mismo puede ser reconstruido a partir de la lista de nodos padres. En este sentido, se crea una lista *Camino* y se parte del nodo objetivo o , el nodo padre es obtenido a partir de `padre[o]`, luego se obtiene el padre de este, y así sucesivamente hasta llegar al nodo de inicio s , luego, la lista es invertida para representar correctamente el orden de tránsito de los nodos.

3.3.2. Algoritmo A*

A diferencia del algoritmo BPA, el algoritmo A* es un algoritmo de búsqueda **informado** que toma en cuenta una **función heurística** $h(n)$ para el cálculo de la función de evaluación, además de la función de costo $g(n)$. La idea detrás de la función heurística, es que representa una estimación del costo del camino que existe entre el nodo n a evaluar y el nodo final. La función de evaluación del nodo n puede ser escrita en este caso como:

$$f(n) = g(n) + h(n)$$

Las funciones heurísticas más utilizadas son la **distancia de Manhattan** y la **distancia euclidiana**. Estas heurísticas suelen ser bastante efectiva en grillas, ya que proporcionan una estimación precisa del costo restante al objetivo sin sobreestimarlos.

El proceso de exploración es exactamente igual que el descrito para el algoritmo BPA, con la única diferencia de que la función de evaluación $f(n)$ ahora también toma en cuenta la función heurística $h(n)$. Una vez el algoritmo halla un camino, el mismo puede ser reconstruido a partir de la lista de nodos padres de la misma forma que para el algoritmo BPA.

3.3.3. Algoritmo A* modificado

El artículo de Xiang Liu et al. [4] propone modificaciones al algoritmo A*, principalmente a su función heurística, y compara la calidad de las soluciones encontradas para laberintos “perfectos”, es decir, sin bucles ni regiones inaccesibles. En dicho artículo, los autores muestran una función heurística modificada cuyo rendimiento es superior a la heurística tradicional del algoritmo A*. Dicha función de evaluación toma la forma:

$$f(n) = g(n) + h(n) + h(j)$$

donde n es el nodo a evaluar y j es el nodo padre de n . La función $g(n)$ sigue representando el costo del camino desde el nodo raíz hasta n , y $h(n)$ es la distancia de Manhattan (o euclidiana) entre n y el nodo objetivo.

En dicho artículo se muestra que la inserción de este término mejora la velocidad de convergencia ya que reduce considerablemente el número de nodos explorados.

La metodología de exploración y de reconstrucción de la solución es idéntica a la de los algoritmos anteriores.

3.4. Generación automática de laberintos: Algoritmo aleatorio de Prim

En cuanto a la generación automática de laberintos, el artículo de Peter Gabrovšek [1] analiza comparativamente seis métodos de generación de laberintos. En este trabajo nos centraremos en el algoritmo más simple de todos: el **algoritmo aleatorio de Prim**.

Los enfoques matemáticos detrás del planteo de dicho algoritmo no se analizan en el presente trabajo, sin embargo, su procedimiento de construcción de laberintos es bastante intuitivo.

1. Inicialización:
 - Se empieza con una grilla donde todas las celdas están bloqueadas por muros.
 - Se selecciona una celda aleatoria y se la marca como parte del laberinto.
 - Se agregan todas sus celdas vecinas (a dos pasos de distancia) a una lista de "fronteras".
2. Expansión del laberinto:
 - Mientras haya celdas en la lista de "fronteras":
 1. Se elige aleatoriamente una celda de la lista.
 2. Se conecta con una celda adyacente que ya esté en el laberinto, eliminando el muro intermedio.
 3. Se marca la celda como parte del laberinto.
 4. Se agregan sus celdas vecinas aún no exploradas a la lista de "fronteras".
 - El proceso se repite hasta que todas las celdas han sido visitadas y el laberinto está completamente generado.

Este algoritmo tiene la particularidad de generar laberintos con caminos largos, con un camino único entre dos puntos.

4. Especificación del agente

En nuestra aplicación particular, nuestro agente será un robot autónomo capaz de llevar a cabo las tareas mencionadas. Para diseñar nuestro agente, partimos de la definición de que un agente es cualquier cosa que es capaz de percibir su entorno con la ayuda de sensores y actuar en ese medio. Sin embargo, generalmente estos agentes poseen un objetivo, es ahí donde nace el concepto de **agente racional** que es aquel que actúa con la intención de alcanzar el mejor resultado esperado. La racionalidad es un concepto ideal de inteligencia: un sistema es racional si "hace lo correcto" en función de su conocimiento.

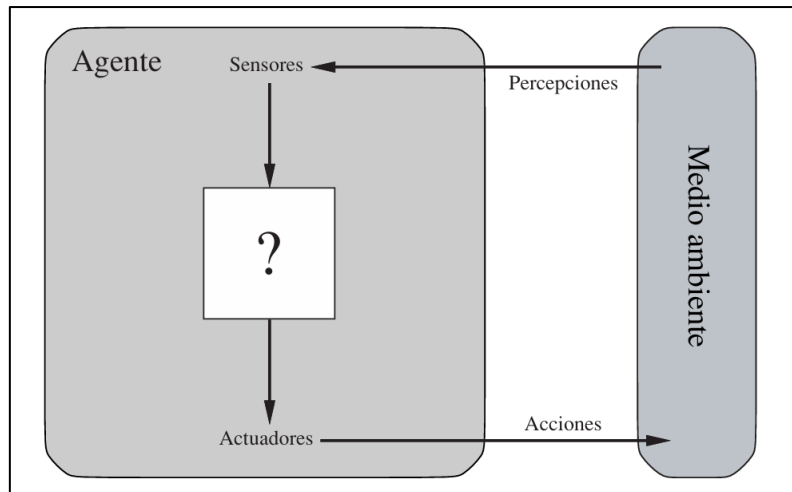


Ilustración 3. Agente y su interacción con el entorno. Fuente: Peter Norvig.

Podemos ver a nuestro sistema como un agente racional, ya que este será capaz de percibir su entorno mediante sensores (por ejemplo, una cámara, un GPS, etc.) y actuar en consecuencia de la mejor manera posible (identificar correctamente el objeto).

Para definir completamente el tipo de agente, es necesario identificar la **naturaleza del entorno** de la forma más completa posible. Para ello, se utilizan las llamadas **tablas REAS** (**R**endimiento, **E**ntorno, **A**ctuadores, **S**ensores). A continuación, se ilustra esta tabla orientada a la aplicación del presente trabajo.

Tipo de agente	Medidas de rendimiento	Entorno	Actuadores	Sensores
Robot que aprende	Reconocer correctamente los objetos (porcentaje de objetos correctamente identificados). Reordenar las cajas con el menor número de pasos posible Llevar las cajas de un punto al otro con el menor recorrido posible	Un galpón en lleno de obstáculos con cajas apiladas que contienen diferentes objetos	Brazos y pinzas para agarrar y mover las cajas Pies o ruedas para transportarse de un punto al otro	Cámara Sensor de cercanía a la posición donde las cajas deben ser depositadas

Tabla 1. Tabla REAS del entorno de trabajo de nuestro agente.

4.1. Propiedades del entorno

- **Observabilidad:** el entorno (galpón o laberinto), en este caso, es **parcialmente observable** para el agente, ya que sus sensores no pueden detectar *a priori* la posición de todos los obstáculos en el camino y actuar en consecuencia de manera de nunca equivocarse de ruta. En nuestro caso, el agente solo podrá ver la posición actual en la que se encuentra y las posiciones adyacentes, por lo que su campo de visión es limitado.

De esta forma, en numerosas ocasiones, el agente podrá llegar a un camino sin salida y tener que comenzar a buscar una ruta alternativa.

- **Determinista o estocástico:** el galpón se trata de un entorno **determinista** puesto que las reglas de movimiento son fijas y siempre producen el mismo efecto. A su vez, no hay eventos aleatorios que afecten al agente (como paredes que aparecen y desaparecen aleatoriamente). Además, la función de costo del agente es consistente y no depende de la aleatoriedad.
- **Episódico o secuencial:** El entorno de la búsqueda del camino en el galpón es **secuencial**, ya que las acciones posteriores dependen del historial de acciones tomadas hasta ese momento. Por ejemplo, si reordena una caja a una posición incorrecta, esto afectará las acciones posteriores para corregirlo. El camino que elige el agente al momento de transportar las cajas también influye en los siguientes movimientos. **La identificación de objetos es episódica.**
- **Estático o dinámico:** En este caso, el entorno es **estático** ya que mientras el robot está ejecutando una tarea, el entorno no cambia.
- **Discreto o continuo:** El entorno es **discreto** puesto que las posiciones del galpón, los datos de entrada (imágenes, posición actual, etc.), la cantidad de instrucciones para reordenar las cajas son magnitudes discretas (número finito de posibilidades).
- **Agente individual o multi-agente:** Como el único agente trabajando es el robot este entorno es de un **agente individual**.

4.2. Estructura del agente

Entre los tipos básicos de programas de agentes se encuentran:

- 4.2.1. **Agentes reactivos simples:** seleccionan las acciones sobre la base de las percepciones actuales, ignorando el resto de las percepciones históricas, es decir, no tienen memoria.
- 4.2.2. **Agentes reactivos basados en modelos:** este tipo de agentes incorporan el concepto de memoria almacenando internamente la información de las partes del mundo que no pueden percibir directamente.
- 4.2.3. **Agentes basados en objetivos:** además de la memoria, estos agentes incorporan algún tipo de información sobre su **meta**, que describa las situaciones deseables.
- 4.2.4. **Agentes basados en utilidad.** Además de la información proporcionada por la meta, la cual permite generalmente encontrar una solución, este tipo de agentes utilizan una función de utilidad que le permite discernir entre dos soluciones, a través de una medida de la “eficiencia” de la solución.

En base a esta información, podemos afirmar que nuestro robot es un **agente basado en objetivos** puesto que:

1. Tiene incorporado el concepto de memoria interna (el agente es capaz de recordar el camino que está recorriendo, el orden de las cajas luego de comenzar a reordenarlas, entre otros).
2. Tiene objetivos específicos, como **reconocer objetos correctamente, reordenar las cajas con el menor número de pasos y llevar cajas con el menor recorrido posible.**

No es un agente basado en utilidad, porque en ningún momento el robot evalúa múltiples soluciones en términos de una función de utilidad más avanzada.

5. Diseño del agente

En esta sección se discute el principio de funcionamiento de los diferentes algoritmos implementados a lo largo del desarrollo del trabajo y que forman parte del programa del agente.

5.1. Etapa 1 - Identificación de objetos mediante visión artificial

La primera etapa del trabajo consistía en la identificación del contenido de 4 cajas diferentes, las cuales se encuentran originalmente apiladas en un orden cualquiera (Ilustración 4), mediante visión artificial.

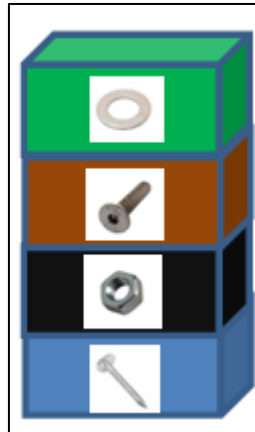


Ilustración 4. Cajas cuyos objetos deben ser identificados.

Para llevar a cabo esta tarea, se desarrolló un agente que permita que permita identificar tornillos, clavos, tuercas y arandelas utilizando los algoritmos KNN y K-Means.

La metodología comienza con la adquisición de imágenes, para lo cual se utilizó la cámara de un teléfono móvil. Las fotografías capturadas corresponden a distintos objetos y fueron almacenadas en una base de datos local. En total, se tomaron 100 imágenes, con 25 fotografías de cada objeto. Las imágenes fueron tomadas con un fondo verde, para facilitar la segmentación del objeto (Ilustración 5).



Ilustración 5. Porción de la base de datos de entrenamiento.

Los objetos no simétricos (clavos y tornillos) fueron **rotados** en las diferentes fotografías para garantizar que la respuesta de nuestro agente no dependa de la rotación del objeto. A su vez, la **iluminación** varía ligeramente entre las diferentes imágenes, lo que aumenta la robustez de las respuestas del agente.

5.1.1. Procesamiento de imágenes

Diferentes operaciones fueron aplicadas secuencialmente a las imágenes originales con el objetivo de obtener imágenes cuya segmentación se vea facilitada. En este sentido, en primer lugar, se detecta el **color medio de la imagen**, que corresponde al fondo de la misma. Luego, los píxeles de la imagen que contengan un color lo suficientemente cercano, se convierten en píxeles negros.

A continuación, la imagen se transforma a **escala de grises** para reducir el número de canales de color de tres (RGB) a uno. La siguiente figura muestra el resultado de esta operación. Podemos constatar que el fondo es detectado correctamente en las imágenes y los objetos ya son fácilmente diferenciables del mismo.

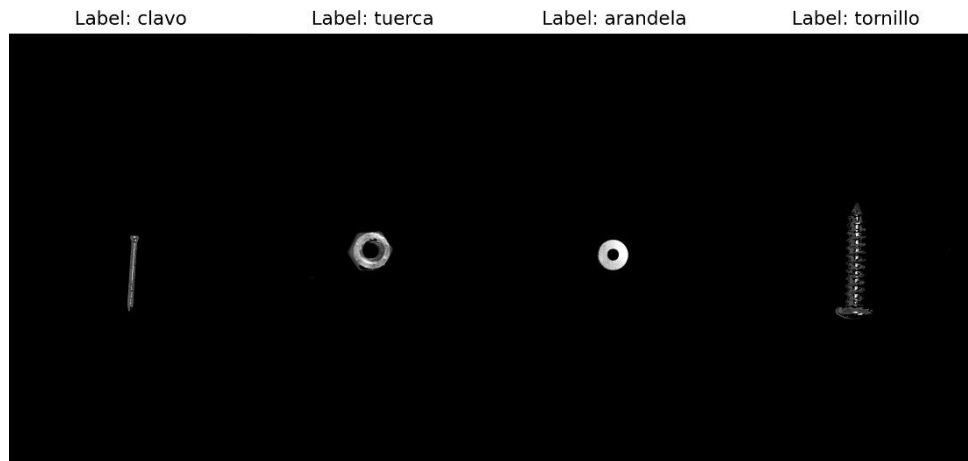


Ilustración 6. Imágenes en escala de grises.

Para eliminar el ruido que puede haber quedado en el fondo de la imagen, se aplica a cada imagen un **filtro de mediana** con un kernel circular de 5x5 píxeles. Este pixel permite suavizar la imagen al reemplazar cada píxel con la mediana de los píxeles en su vecindario.

Luego, para facilitar la utilización de filtros y de diversas funciones más avanzadas (búsqueda de contornos, entre otras), se **binariza** la imagen a blanco y negro. El threshold de binarización es hallado automáticamente para cada imagen utilizando el **algoritmo de Li**, el cual produjo buenos resultados para la gran mayoría de nuestras imágenes.

Finalmente, se aplica un **filtro de cierre** a las imágenes empleando un kernel circular de 10x10. Este filtro realiza una operación de dilatación seguida de una erosión. Lo que es útil para cerrar pequeños agujeros oscuros en los objetos debido al ruido o a la iluminación en la imagen (Ilustración 7). El tamaño del kernel determina el tamaño de los agujeros que se pueden cerrar.

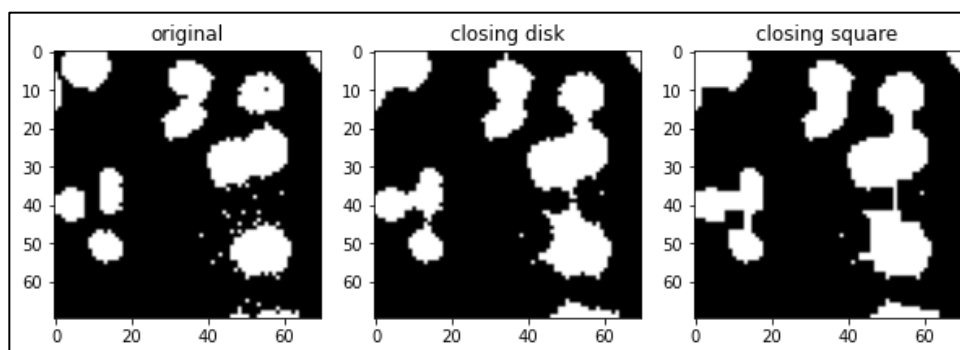


Ilustración 7. Filtro de cierre.

En la siguiente figura se ilustran algunas de las imágenes ya procesadas. Ahora, procederemos a describir el proceso de detección de contornos

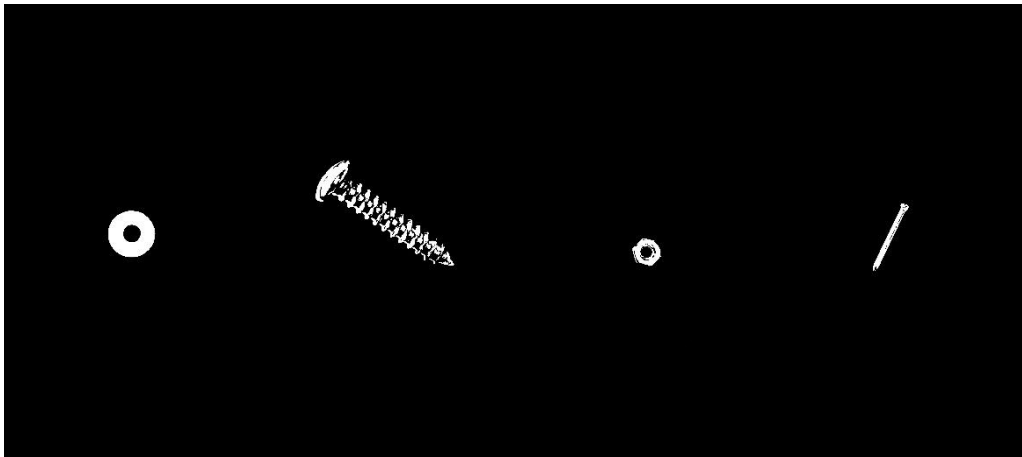


Ilustración 8. Imágenes procesadas.

5.1.2. Detección de contornos

Antes de proceder a la extracción de características de la imagen, se buscan los **contornos externos** de los objetos utilizando la función ***findContours()*** de OpenCV. Esta función puede devolver numerosos contornos, pero como nos interesa obtener únicamente el contorno externo más representativo del objeto, filtramos los demás en función del área encerrada, considerando únicamente el de mayor área. La siguiente figura muestra los resultados de esta operación, con los contornos detectados en rojo.

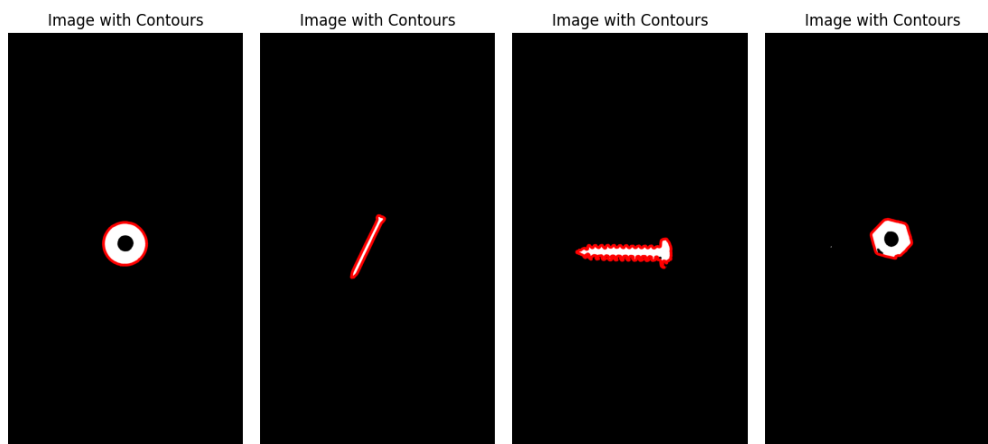


Ilustración 9. Imágenes con sus contornos externos detectados.

Luego, a partir de los contornos detectados se calcula el **Convex Hull** (envolvente convexa, en español), que nos da el menor polígono convexo que engloba los puntos del contorno. En términos geométricos, podemos pensarlo como si tenemos un conjunto de puntos en un plano, su envoltura convexa es como si tomaras una banda elástica y la estiraras alrededor de todos los puntos; cuando la sueltas, la banda se ajusta formando un polígono convexo que los encierra. La envolvente convexa se calculó para cada imagen utilizando la función ***convexHull()*** de OpenCV.

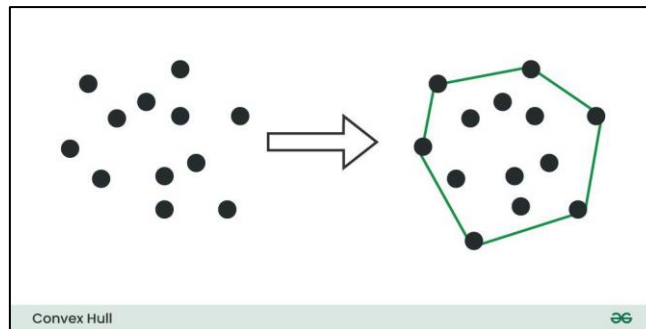


Ilustración 10. Envoltente convexa.

Se comprobó que los contornos pueden verse afectados por ruido en la imagen, pequeñas variaciones de píxeles, entre otros, mientras que la envoltura convexa suaviza estos efectos porque ignora pequeñas concavidades y se enfoca en la estructura general del objeto. De la misma forma, si queremos calcular propiedades del objeto como su centroide, área o perímetro de manera estable, la envoltura convexa proporciona en general una forma más limpia y fácil de trabajar.

Por consiguiente, las imágenes son recortadas alrededor de las **envoltentes convexas calculadas en lugar de los contornos**. La siguiente figura ilustra los resultados.

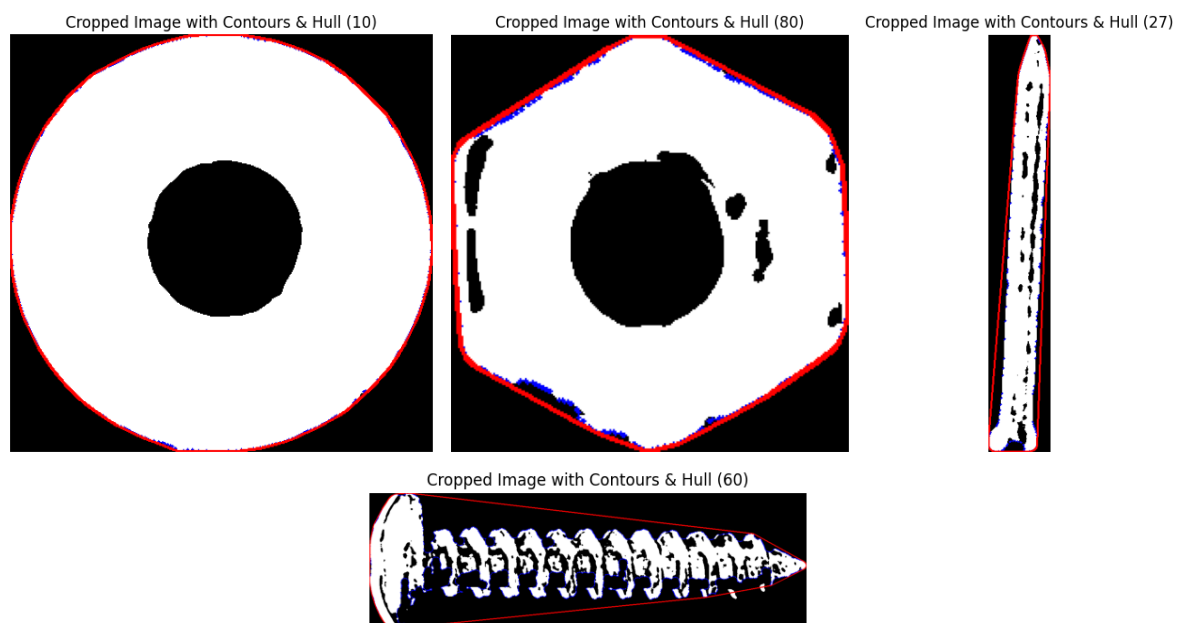


Ilustración 11. Imágenes recortadas alrededor de la envoltente convexa.

5.1.3. Extracción de características

En esta etapa se detalla el proceso de extracción de características útiles que permitan discernir entre un tipo de objeto y otro. Durante esta etapa, se dio prioridad a aquellas características que sean **invariables frente a la rotación y escala del objeto**, es decir, que cuyo valor no sea considerablemente afectado por el “zoom” de la cámara y el tamaño del objeto, ni por su orientación.

En un primer momento, se comenzó a trabajar con **características de forma** del objeto. La forma de cada objeto es bastante diferente con lo cual esto resultó ser un buen punto de partida. Por ejemplo, para extraer la **circularidad** del contorno, se calculó un índice de circularidad dado por la siguiente fórmula:

$$Circularidad = 4\pi \frac{Area}{Perímetro^2}$$

Además, se ideó un índice similar dado por la relación entre el área encerrada por el contorno y el área del menor círculo que encierra a dicho contorno. Valores cercanos a la unidad de cualquiera de estos índices indica un alto grado de circularidad.

$$Ratio\ de\ area\ circular = \frac{Area}{Area\ del\ menor\ círculo\ que\ encierra\ el\ contorno}$$

También se calcularon otras relaciones de forma llamadas **solidez** y **convexidad**, definidas respectivamente como:

$$Solidez = \frac{Area}{Area\ de\ la\ envolvente\ convexa}$$

$$Convexidad = \frac{Perímetro\ de\ la\ envolvente\ convexa}{Perímetro}$$

Para detectar objetos alargados, se calculó la relación entre la longitud de los ejes principales de la imagen. Aquellos objetos alargados tienen sus ejes principales de longitudes muy diferentes, por los que esta relación se aleja de la unidad.

$$Relación\ de\ aspecto = \frac{Longitud\ eje\ menor}{Longitud\ eje\ mayor}$$

Otra característica que da una medida de la elongación de los objetos se denomina **excentricidad** y es calculada como:

$$Excentricidad = \sqrt{1 - \left(\frac{Longitud\ eje\ menor^2}{Longitud\ eje\ mayor^2} \right)}$$

Los objetos simétricos tienen una excentricidad cercana a cero, mientras que la excentricidad se aproxima a la unidad para los objetos alargados.

También se calcularon los **momentos de Hu** de la imagen, de los contornos y de la envolvente convexa, en este sentido se observaron diferencias muy significativas según a la entidad (imagen, contornos o envolvente) con la que se calculaban los momentos.

5.1.4. Selección de características

Todas las características que se mencionan en la sección anterior fueron calculadas para todas las imágenes. Estas contienen la información de la forma de los objetos a identificar, sin embargo, esto no quiere decir que debamos utilizarlas a todas para clasificar nuestros objetos.

Puede suceder, por ejemplo, que el rango de valores que toma habitualmente una determinada característica no sea muy diferente para nuestro conjunto de objetos, con lo cual, esta característica no sirve para diferenciarlos.

En este sentido, se graficaron los **gráficos de caja** para cada característica en función del tipo de objeto a identificar. Aquellas características cuyo rango de valores se encontraba separado para las diferentes categorías fueron seleccionadas. La siguiente ilustración algunos de los diferentes gráficos de cajas.

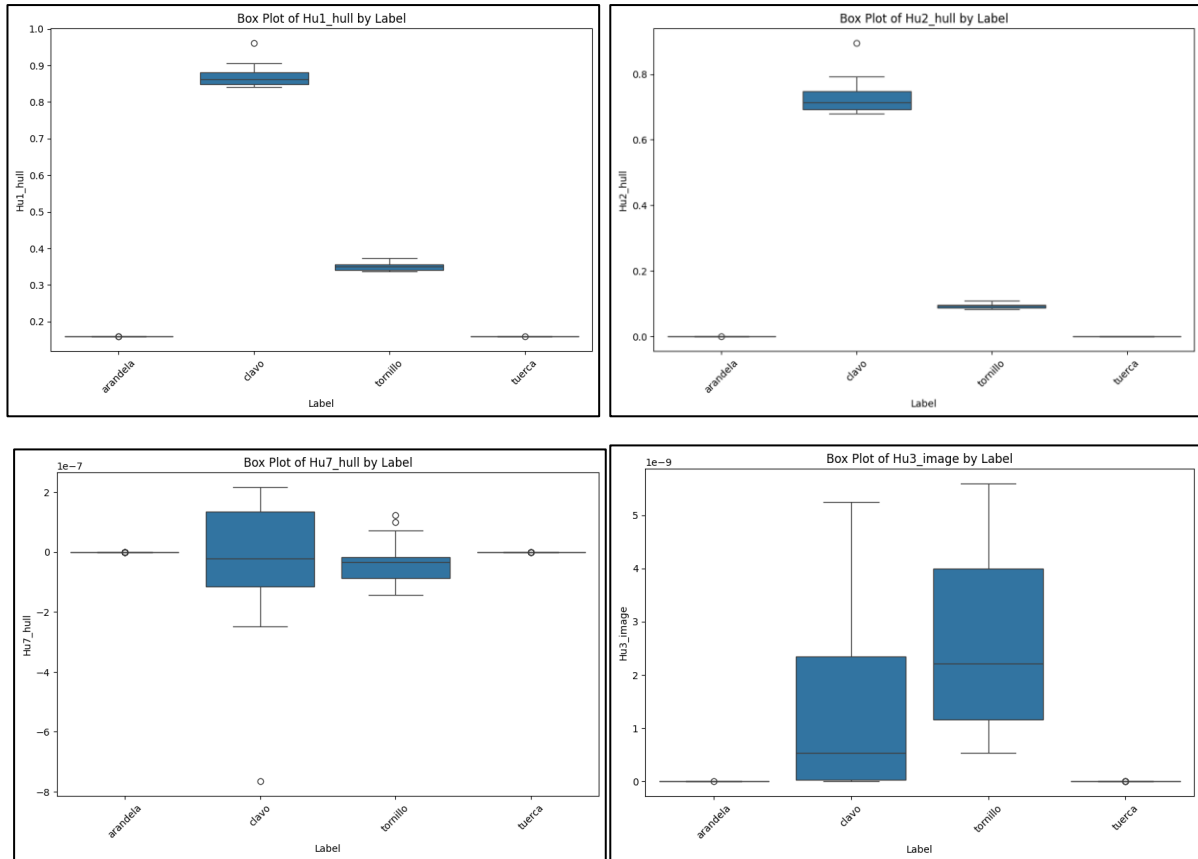


Ilustración 12. Gráficos de caja de algunas de las características.

Vemos, por ejemplo, que las dos características superiores son bastante útiles para separar un clavo de un tornillo, pero, por otro lado, no permiten distinguir muy bien una arandela de una tuerca. Por otra parte, **las dos características inferiores son bastante mediocres para clasificar los objetos y no deberían ser utilizadas como entradas de los algoritmos de identificación de objetos.**

Esta revisión y metodología fue realizada para cada una de las características calculadas anteriormente y, finalmente, se eligieron tres de ellas. Las **características elegidas** fueron:

- **Primer momento de Hu de la envolvente convexa del objeto.**
- **Ratio de área circular.**
- **Excentricidad.**

A continuación, se muestran los gráficos de cajas de las mismas:

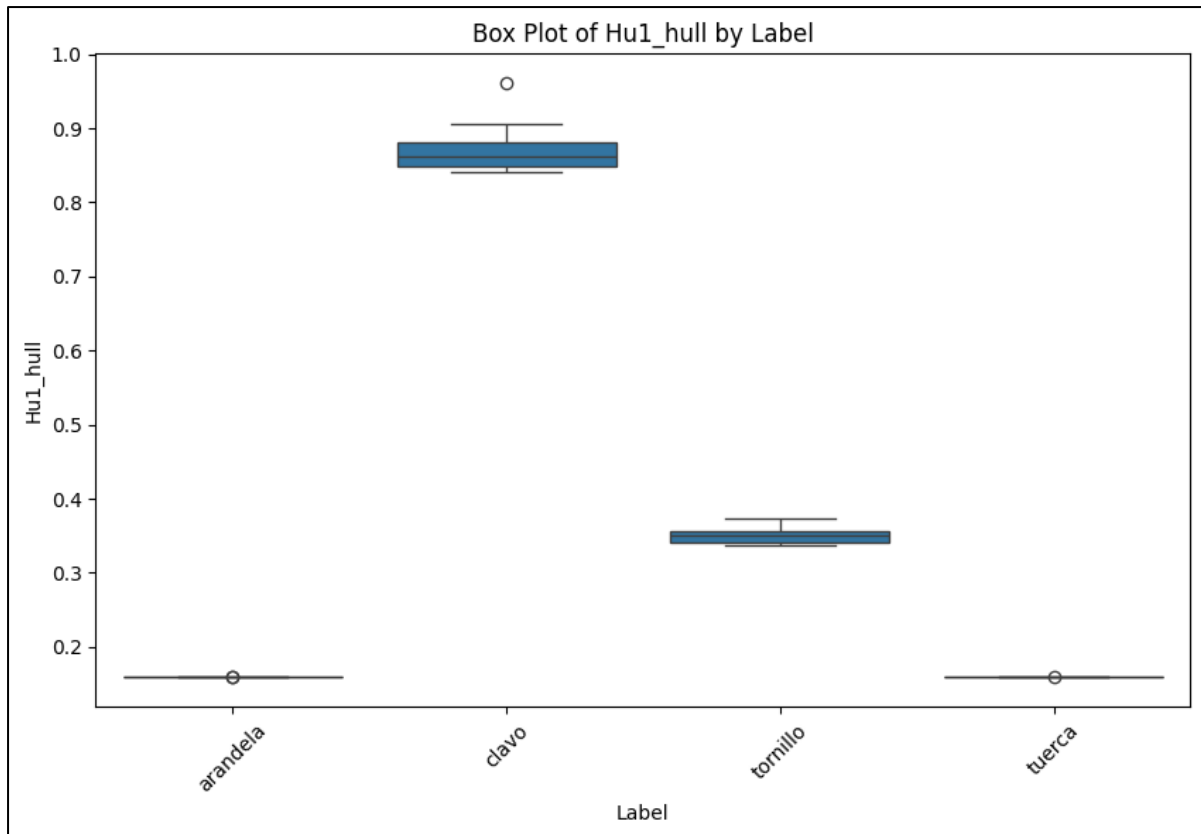


Ilustración 13. Gráfico de cajas para la primera característica seleccionada.

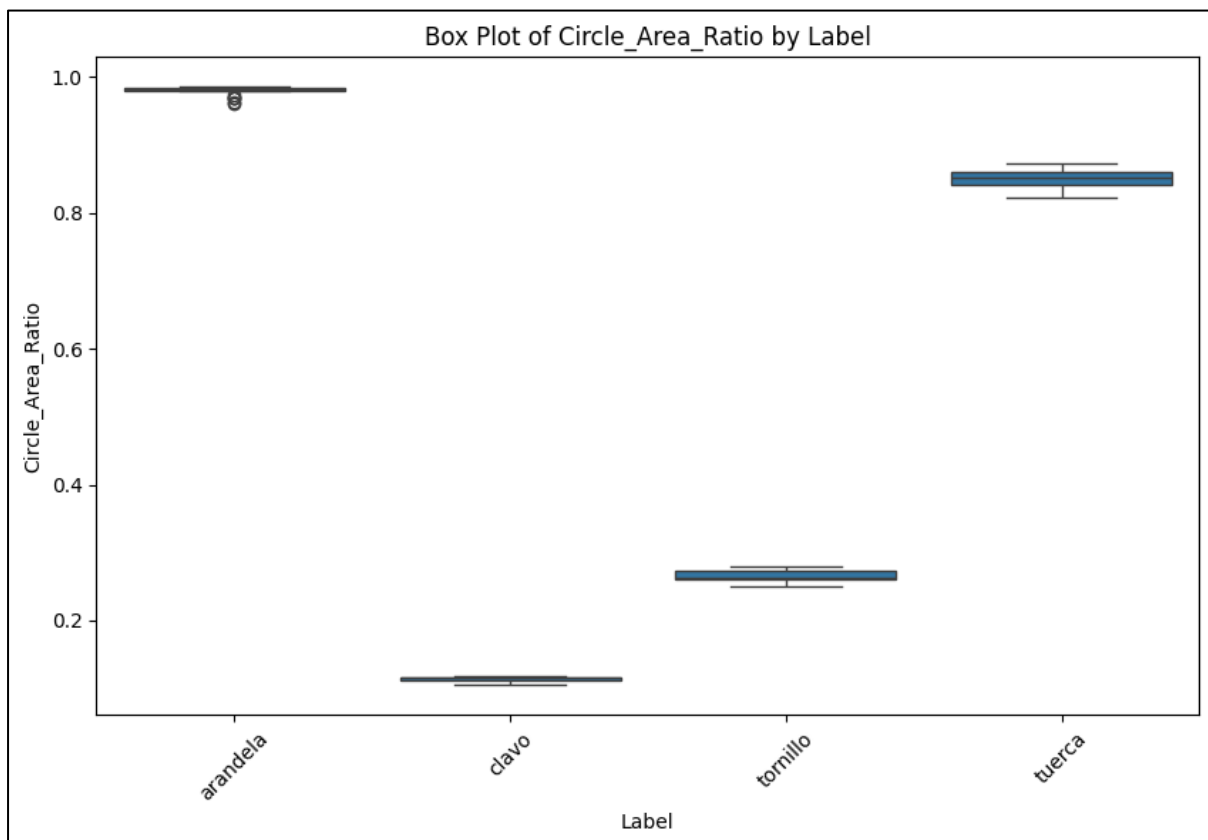


Ilustración 14. Gráfico de cajas para la segunda característica seleccionada.

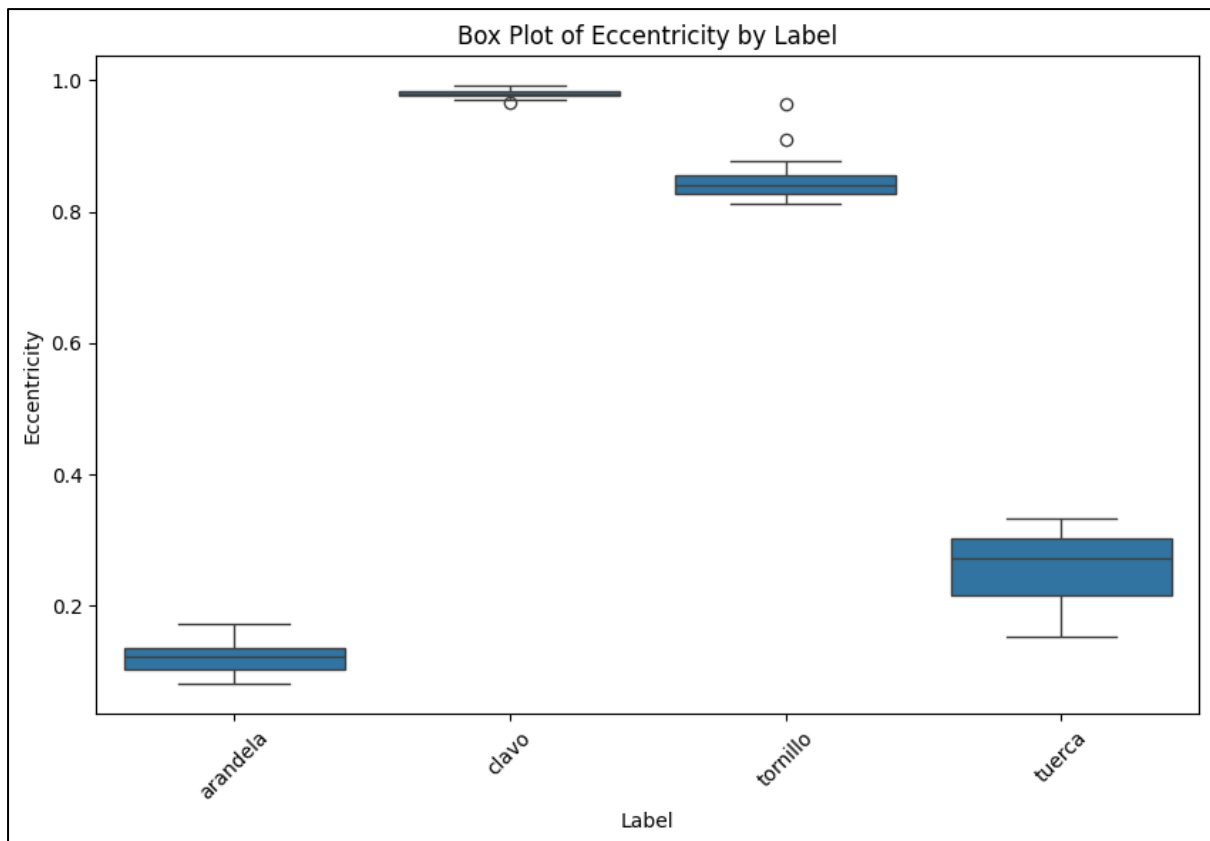


Ilustración 15. Gráfico de cajas de la tercera característica seleccionada.

Podemos constatar que **el rango de valores que toman las características seleccionadas difiere considerablemente según la categoría de objeto a identificar.**

5.1.5. Procesamiento de las características

Antes de emplear estas características seleccionadas como entrada de los algoritmos de identificación de objetos, modificaremos las magnitudes de todas ellas a la misma escala para evitar diferencias significativas de magnitud que afecten al rendimiento de los algoritmos.

En este sentido, un escalador ***StandardScaler()*** de la librería scikit-learn fue empleado.

5.1.6. Algoritmo kNN

Como se menciona en la sección 3.1.2, el **algoritmo kNN** es un algoritmo supervisado de clasificación que emplea la proximidad en base a una métrica de distancia (generalmente euclidiana) para realizar predicciones acerca de la agrupación de un punto de datos individual frente a un conjunto de datos.

El parámetro más importante a definir es k , el número de vecinos a considerar para determinar la predicción. La predicción será igual a la clase correspondiente a la mayoría de los k vecinos más cercanos. En general, se utilizan valores impares de k para evitar empates, de igual forma, k debe ser mayor a la cantidad de clases diferentes a predecir, en este caso $k > 4$.

Experimentalmente se determinó, mediante técnicas de validación cruzada, que **$k=7$** proporciona resultados satisfactorios para nuestra aplicación.

Los pasos realizados por el algoritmo son los siguientes:

1. El proceso se inicia cargando un conjunto de datos de entrenamiento, en formato CSV, el cual incluye las características y etiquetas de clase asociadas.
2. A continuación, se especifica el número de vecinos más cercanos, k , que se utilizarán para clasificar el punto de datos en cuestión; en este caso $k=7$.
3. Luego, se procede a calcular la distancia euclidiana entre el punto de datos de entrada y todos los puntos de datos presentes en el conjunto de entrenamiento.
4. Posteriormente, se seleccionan los k puntos de datos más cercanos al punto de datos de entrada del conjunto de entrenamiento.
5. Después, se determina la etiqueta de clase del punto de datos de entrada contando las etiquetas de clase de los k vecinos más cercanos. La etiqueta de clase asignada al punto de datos de entrada es la etiqueta más frecuente entre estos vecinos más cercanos.
6. Finalmente, se devuelve la etiqueta de clase calculada como la predicción de clase para el punto de datos de entrada.

Como cálculo adicional, se incluyó una métrica de la confianza del algoritmo en su predicción, dada por:

$$\text{Confianza} = \frac{\text{Total de votos correctos}}{\text{Total de votos}}$$

La siguiente figura muestra una predicción para una arandela. Vemos que **el algoritmo predice correctamente el objeto con una confianza del 100%**.

Identificación de objetos

Cargar imagen

Algoritmo de reconocimiento:

☐ K-Means ☒ KNN

Ejecutar algoritmo

Clase resultante: arandela

Confianza: 100.00%

Imagen procesada junto a la envolvente convexa

Características extraídas

Hu_Moment_1: 0.16	Scaled_Hu_Moment_1: -0.78
Circle_Area_Ratio: 0.98	Scaled_Circle_Area_Ratio: 1.16
Eccentricity: 0.12	Scaled_Eccentricity: -1.17

Ilustración 16. Ejemplo de kNN para una arandela.

La siguiente representación en 3D muestra que efectivamente las características de la imagen de entrada se encuentran más cerca del grupo correspondiente a las arandelas, lo cual justifica la confianza del 100%.

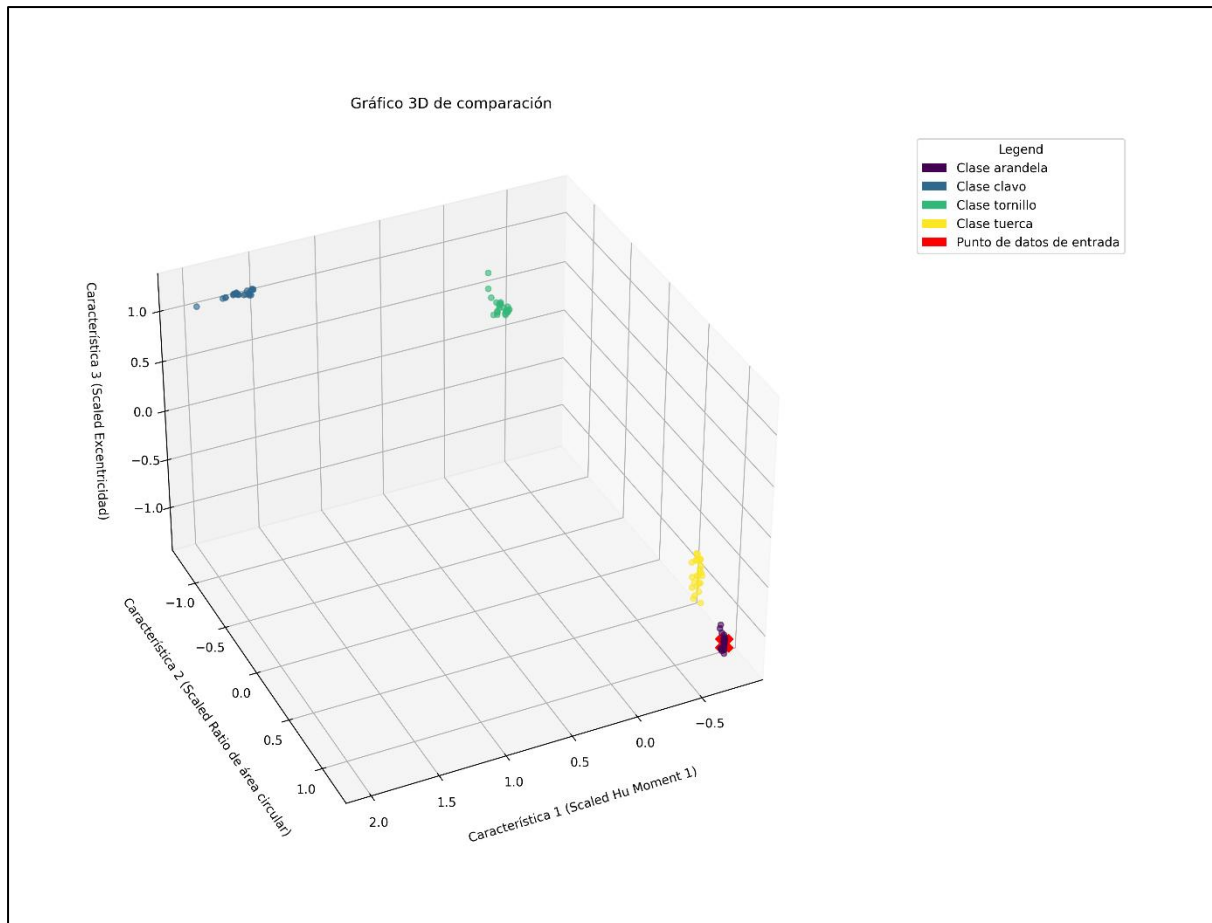


Ilustración 17. Representación 3D del algoritmo KNN.

5.1.7. Algoritmo K-Means

K-Means es un algoritmo de aprendizaje no supervisado empleado para la **agrupación de datos**. Los datos de entrenamiento utilizados por este algoritmo no están etiquetados, lo que quiere decir que el algoritmo no posee ningún tipo de información previa acerca de la estructura de clasificación del conjunto de datos.

El principio de funcionamiento se basa en un **agrupamiento iterativo basado en centroides**, que divide un conjunto de datos en diferentes grupos o clusters en función de la distancia euclidiana entre cada punto y los centroides. Los puntos de datos se agrupan dentro de la misma categoría que su centroide más cercano. El centroide es la media de todos los puntos dentro del cluster.

El principal hiperparámetro a elegir es k , el número de clusters en el que se dividirán el conjunto de datos. En este caso, como sabemos a priori que hay cuatro tipos de objetos diferentes, se escoge **$k=4$** .

Se utilizó igualmente la metodología de **inicialización kmeans++**, en la cual se optimiza la elección de los centroides iniciales utilizando una medida de probabilidad de selección según la distancia.

En este sentido, el primer paso de este algoritmo de inicialización es elegir el primer centroide al azar de entre los puntos del conjunto de datos. Para cada punto restante, se calcula la distancia al centroide más cercano. Luego, se selecciona el siguiente centroide con **probabilidad proporcional al cuadrado de su distancia** al centroide más cercano. Esto significa que los puntos más alejados tienen

más probabilidad de ser elegidos. Estos dos últimos pasos se repiten hasta seleccionar k centroides en total, en este caso 4 centroides.

Luego, cada punto del conjunto de datos se asigna al clúster cuyo centroide esté más cercano. Para lograrlo, se calcula la distancia euclidiana entre cada punto y cada centroide, y se asigna el punto al clúster cuyo centroide tenga la menor distancia.

Una vez que todos los puntos han sido asignados a un clúster, se recalculan los centroides de cada clúster como la media de todos los puntos del clúster. Esto actualiza la posición del centroide para reflejar la nueva agrupación.

Se repiten estos dos últimos pasos hasta que los centroides de los clústeres ya no cambien o hasta que se alcance el número máximo de iteraciones establecido. En cada iteración se calcula la **inercia** del agrupamiento resultante.

Todo el proceso es repetido un n_{init} veces para evitar que las condiciones iniciales afecten al resultado encontrado. El mejor agrupamiento (menor inercia) es conservado y devuelto al final del proceso. Finalmente, se revisa en qué clúster quedó nuestro objeto en función de la distancia al centroide más cercano y se devuelve como resultado el clúster al que fue asignado.

La precisión del agrupamiento es calculada al final como:

$$\text{Precisión de agrupamiento} = \frac{\text{Total de puntos de datos correctamente agrupados entre sí}}{\text{Total de puntos de datos}}$$

La siguiente figura muestra un ejemplo de aplicación para un tornillo:

Identificación de objetos

Cargar imagen



Algoritmo de reconocimiento:

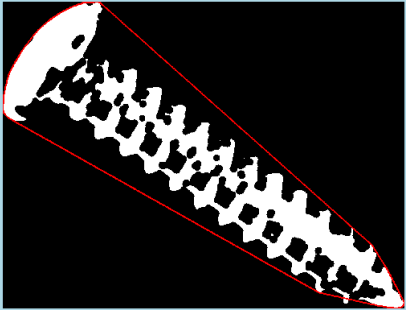
☒ K-Means
 ☐ KNN

Ejecutar algoritmo

Grupo resultante: 2 - Clase: tornillo

Precisión del agrupamiento: 100.00%

Imagen procesada junto a la envolvente convexa



Características extraídas

Hu_Moment_1: 0.34	Scaled_Hu_Moment_1: -0.16
Circle_Area_Ratio: 0.27	Scaled_Circle_Area_Ratio: -0.76
Eccentricity: 0.83	Scaled_Eccentricity: 0.74

Ilustración 18. Ejemplo de K-Means para un tornillo.

Si vamos a la representación 3D del algoritmo, vemos que el tornillo se parece realmente a los puntos de datos del grupo 2, el cual correctamente tiene ligada la clase “tornillo”. A su vez, podemos observar que objetos de diferente clase no se encuentran agrupados en el mismo cluster, lo que justifica la precisión del 100% que se ve en la imagen anterior.

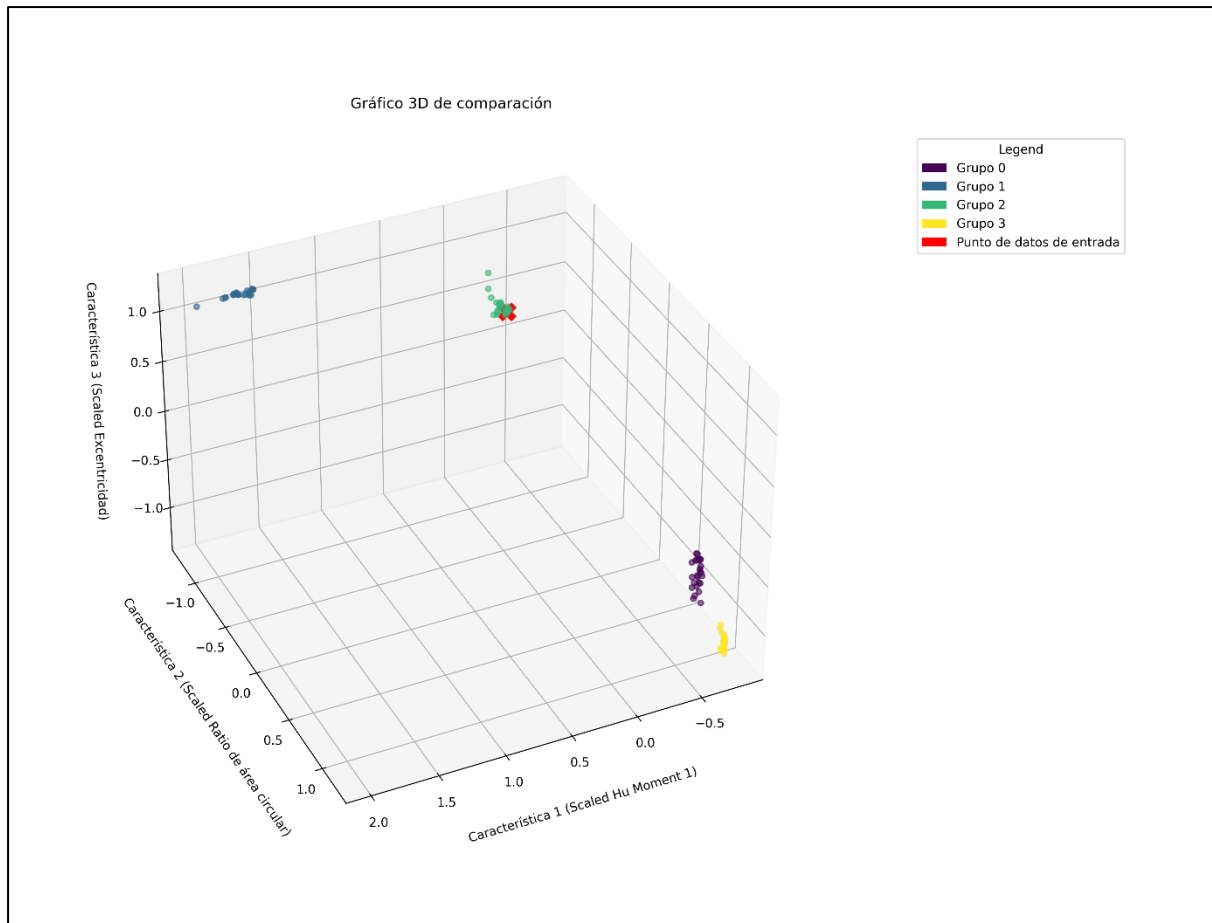


Ilustración 19. Representación 3D del algoritmo K-Means.

5.2. Etapa 2 - Planificación y reordenamiento de las cajas

La **planificación** es el proceso de búsqueda y articulación de una secuencia de acciones que permitan alcanzar un objetivo. En esta etapa se busca que el robot reordene una pila de 4 cajas en donde cada caja contiene un grupo de objetos identificado a partir de la etapa anterior. Para ello, se formula el problema mediante el lenguaje **STRIPS**.

STRIPS modela problemas de planificación a través de una representación lógica en la que:

- Los **estados** se describen como conjuntos de **predicados**.
- Los **predicados** son expresiones lógicas que describen la relación entre los objetos y ciertas condiciones del mundo.
- Las **acciones** representan las operaciones o movimientos que un agente puede realizar en un entorno durante la ejecución de un plan. Estas **acciones** tienen **precondiciones** (requisitos para ejecutarlas) y **efectos** (cambios en el estado del mundo tras su aplicación).
- Los **objetivos** son condiciones que el sistema debe alcanzar.

Para ejecutar una secuencia de planificación, es necesario definir un **estado inicial**, dado por una secuencia de predicados que se consideran verdaderos en ese momento.

En nuestro caso, hemos definido un **dominio de planificación** llamado *CAJAS_domain* para abordar el problema del reordenamiento de las cajas. En este dominio, se definieron 5 predicados principales basándose en el famoso problema de planificación *Blocks-world*. Estos predicados son:

- **SOBRE ?x ?y**: indica si la caja x está colocada sobre la caja y.
- **LIBRE ?x**: indica si la caja x está libre (puede ser agarrada).
- **SOSTENER ?x**: indica si la mano o pinza está sosteniendo la caja x.
- **ENMESA ?x**: indica si la caja x se encuentra apoyada sobre la mesa.
- **MANOLIBRE**: indica si la mano o pinza está libre (no está agarrando ninguna caja).

Por otra parte, en el dominio se definieron 4 acciones:

- AGARRAR
- BAJAR
- APILAR
- DESAPILAR

Cada una de estas de estas acciones tiene ciertas precondiciones que cumplir antes de su ejecución y ciertos efectos que modifican el estado del mundo tras ser ejecutadas.

Para resolver un problema particular de planificación es necesario definir un **problema**. En nuestro caso, cada vez que se ejecuta el algoritmo de búsqueda de soluciones de planificación, se crea un archivo de problema llamado **CAJAS_problem**, el cual contiene básicamente las condiciones iniciales y el estado objetivo. Tanto las condiciones iniciales como el estado objetivo se obtienen automáticamente desde la interfaz de usuario que se muestra en la siguiente figura.

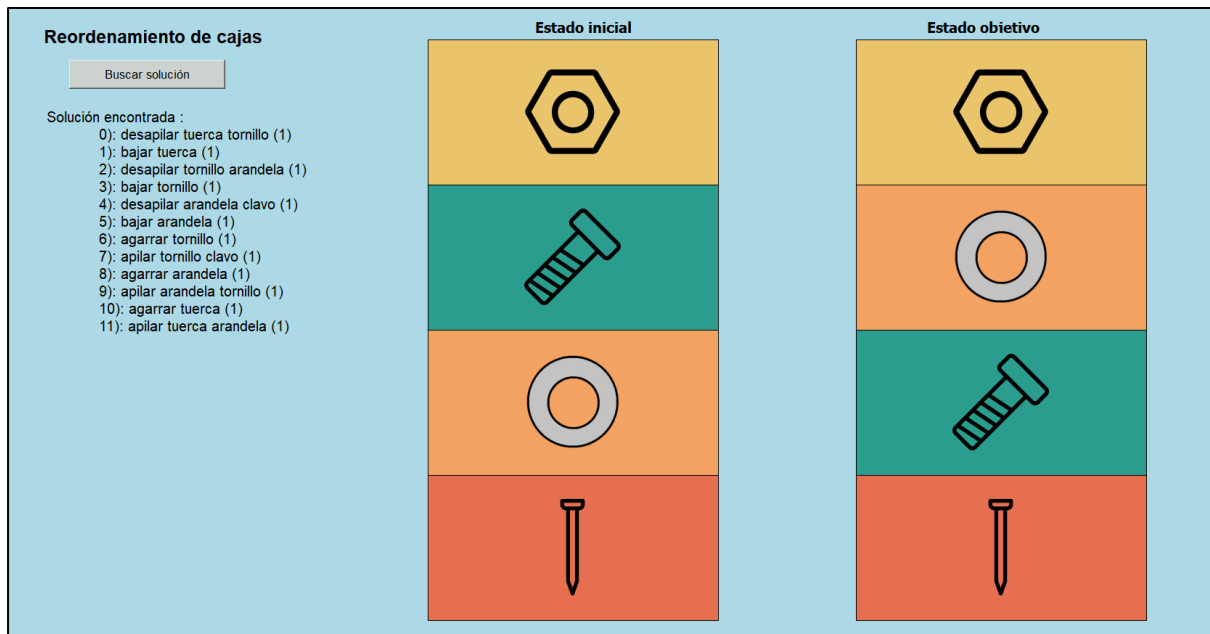


Ilustración 20. Interfaz de usuario para el reordenamiento de las cajas.

Cuando el usuario da click sobre el botón de “Buscar solución”, el archivo de problema es creado y se llama localmente a un *solver* de planificación obtenido desde la página web <http://lcas.lincoln.ac.uk/fast-downward/>.

Vemos entonces que se ha ideado un enfoque efectivo para abordar el problema de apilamiento de cajas utilizando técnicas de planificación basadas en STRIPS. La solución logra satisfacer el objetivo del problema al reorganizar las cajas en el nuevo orden de apilamiento deseado.

5.3. Etapa 3 - Transporte de las cajas

En la última etapa del trabajo, el robot debe ser capaz de transportar las cajas desde un punto A hasta un punto B de un galpón. Para mostrar el potencial de los algoritmos de búsqueda implementados, se pensó al galpón como una grilla de 40x40 casillas y se implementó un algoritmo de **generación de laberintos** (ver sección 3.4). De igual forma, el usuario también tiene la posibilidad de colocar los obstáculos en el terreno utilizando el click izquierdo del mouse.

Se analizaron, a su vez, tres algoritmos de búsqueda diferentes:

- Algoritmo A*
- Búsqueda primero en anchura
- Algoritmo A* modificado

En esta sección nos centraremos en el **algoritmo A***, ya que es el que fue solicitado en las consignas del trabajo, sin embargo, el principio de funcionamiento de todos ellos se detalla en la sección 3.3. **El funcionamiento del resto de algoritmos es similar, cambiando únicamente la función de evaluación utilizada por el mismo.**

El **algoritmo A*** es un algoritmo de búsqueda informado que toma en cuenta una **función heurística** $h(n)$ para el cálculo de una **función de evaluación** $f(n)$, además de una **función de costo** $g(n)$.

$$f(n) = h(n) + g(n)$$

donde, n es una casilla a evaluar.

La idea detrás de la función de evaluación es poder explorar prioritariamente aquellas casillas que prometen acercarse a la solución. La función de costo $g(n)$ **denota el costo acumulado de la ruta para llegar a n a partir del nodo de inicio**. En nuestro caso, cada casilla visitada tiene un **costo unitario**, por lo que **$g(n)$ es igual al número de casillas visitadas para ir desde el nodo de inicio hasta n** .

Por otro lado, la función $h(n)$ representa una estimación del costo del camino que existe entre el nodo n a evaluar y el nodo final.

El algoritmo explorará su entorno dando prioridad a aquellas celdas con menor valor de f , lo que implica que la utilización de la heurística permite al algoritmo guiar su búsqueda hacia la solución de manera eficiente.

Las métricas de distancia más utilizadas son la distancia de Manhattan y la distancia euclidiana, nosotros optaremos por la primera. La **distancia de Manhattan** en el contexto de un laberinto 2D está dada por:

$$d_{Manhattan} = |x_2 - x_1| + |y_2 - y_1|$$

donde x_i e y_i pueden ser pensados como el número de fila y de columna de una casilla, respectivamente.

El algoritmo A* mantiene dos listas esenciales:

- **Lista abierta:**
 - Contiene nodos que deben evaluarse.
 - Ordenados por valor de $f(n)$ (primero el más bajo).
 - Se añaden nuevos nodos a medida que se descubren.

- **Lista cerrada:**
 - Contiene nodos ya evaluados
 - Ayuda a evitar la reevaluación de nodos
 - Se utiliza para reconstruir la trayectoria final

A continuación, se detalla el funcionamiento paso a paso del algoritmo A*:

1. Se selecciona el nodo de inicio y el nodo objetivo.
2. Se inicializa la **lista abierta** como una cola de prioridad. La lista abierta comienza sólo con el nodo inicial. A su vez, se inicializa la **lista cerrada** como una lista vacía.
3. Se asocia al nodo inicial con un costo acumulado nulo ($g(0) = 0$).
4. Mientras la lista abierta no esté vacía:
 1. Se selecciona el nodo de la lista abierta con el valor de la función de evaluación más bajo (costo acumulado + estimación heurística).
 2. Si el nodo seleccionado es el estado objetivo, se ha encontrado el camino óptimo y se termina el algoritmo.
 3. De lo contrario,
 1. Se añade el nodo actual a la lista cerrada
 2. Se revisan los vecinos del nodo actual y se calculan sus valores de función de evaluación.
 3. Se asocia al nodo actual como nodo padre de los nodos vecinos (para luego reconstruir la ruta).
 4. Los vecinos del nodo actual se añaden a la lista de prioridad en base a estos valores de $f(n)$.
5. Si la frontera se vacía y no se ha encontrado el nodo objetivo, significa que no hay un camino posible.

Una vez el algoritmo halla un camino, el mismo puede ser reconstruido a partir de la lista de nodos padres. En este sentido, se crea una lista *Camino* y se parte del nodo objetivo o , el nodo padre es obtenido a partir de `padre[o]`, luego se obtiene el padre de este, y así sucesivamente hasta llegar al nodo de inicio s , luego, la lista es invertida para representar correctamente el orden de tránsito de los nodos.

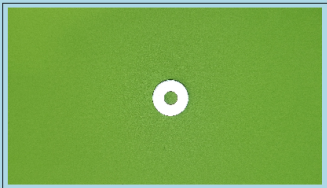
6. Ejemplo de aplicación

En esta sección se muestran ejemplos de aplicación de cada una de las etapas del proyecto. En primer lugar, comenzaremos con la etapa de reconocimiento de objetos, donde se mostrarán los resultados de predicción para la misma imagen (arandela) utilizando los algoritmos K-means y KNN.

Vemos en las figuras 21 y 22 que ambos algoritmos predicen correctamente la clase de objeto presente en la imagen (arandela).

Identificación de objetos

Cargar imagen



Algoritmo de reconocimiento:

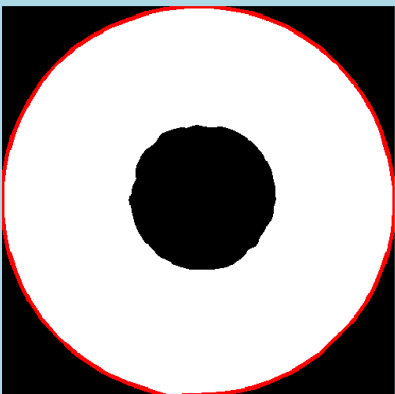
☐ K-Means ☒ KNN

Ejecutar algoritmo

Clase resultante: arandela

Confianza: 100.00%

Imagen procesada junto a la envolvente convexa



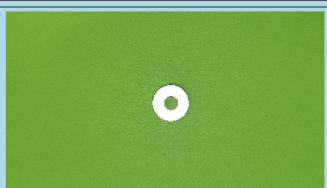
Características extraídas

Hu_Moment_1: 0.16	Scaled_Hu_Moment_1: -0.78
Circle_Area_Ratio: 0.98	Scaled_Circle_Area_Ratio: 1.17
Eccentricity: 0.12	Scaled_Eccentricity: -1.16

Ilustración 21. Ejemplo de aplicación (KNN).

Identificación de objetos

Cargar imagen



Algoritmo de reconocimiento:

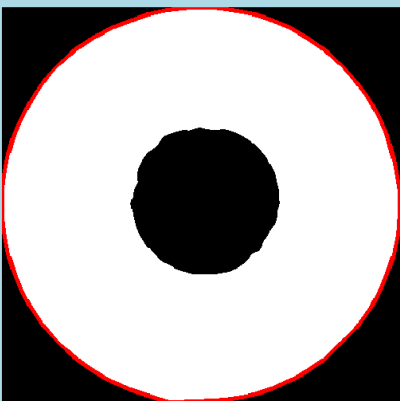
☒ K-Means ☐ KNN

Ejecutar algoritmo

Grupo resultante: 1 - Clase: arandela

Precisión del agrupamiento: 100.00%

Imagen procesada junto a la envolvente convexa



Características extraídas

Hu_Moment_1: 0.16	Scaled_Hu_Moment_1: -0.78
Circle_Area_Ratio: 0.98	Scaled_Circle_Area_Ratio: 1.17
Eccentricity: 0.12	Scaled_Eccentricity: -1.16

Ilustración 22. Ejemplo de aplicación (K-means).

Luego, procedemos a mostrar un ejemplo de reordenamiento de cajas, los ordenes inicial y final fueron escogidos al azar para ilustrar el ejemplo. Al seguir la solución hallada podemos pasar efectivamente desde el ordenamiento inicial hasta el final.

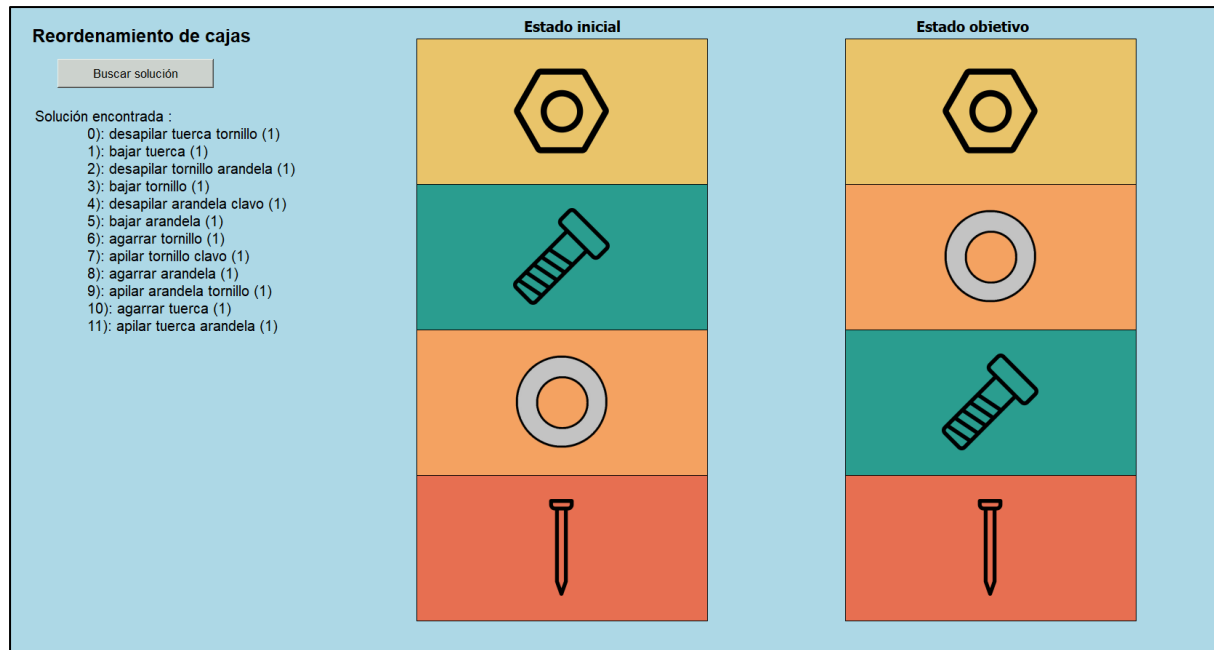


Ilustración 23. Ejemplo de aplicación (Planificación y reordenamiento).

Finalmente, mostramos un caso de aplicación de la búsqueda de rutas y transporte de cajas utilizando el algoritmo A*. Las celdas amarillas representan el camino más óptimo para ir desde el punto verde (inicio) hasta el rojo (final). Las celdas grises representan las celdas visitadas, y las azules son las celdas que se encuentran en la lista de *frontera* (*openSet*).

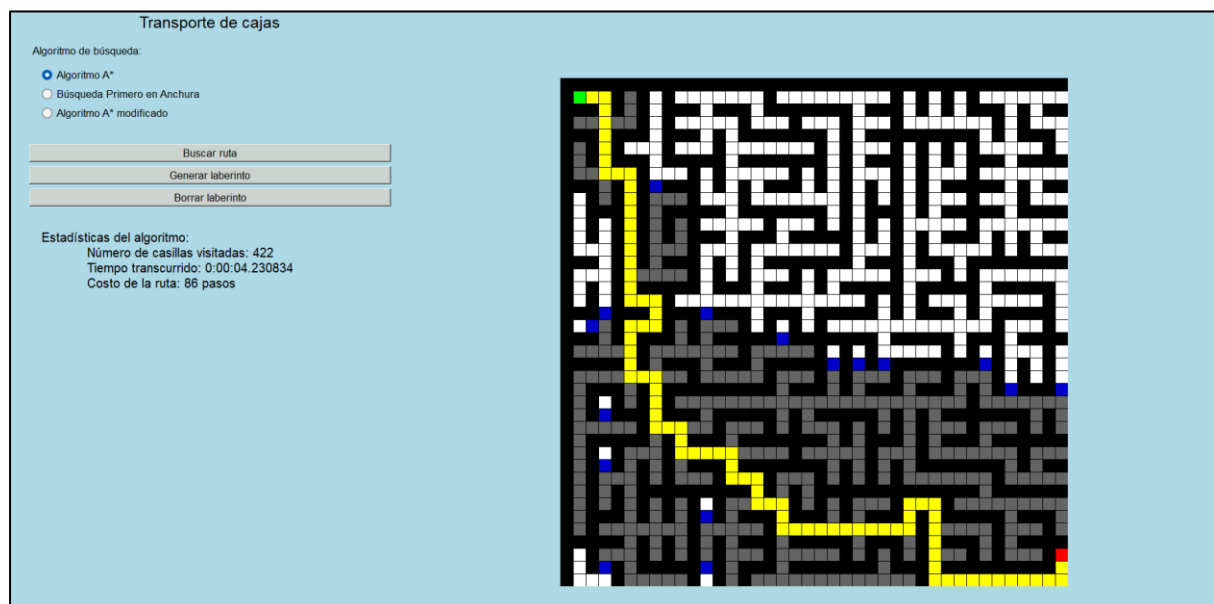


Ilustración 24. Ejemplo de aplicación (Búsqueda de rutas).

7. Resultados

En la primera etapa, ambos algoritmos (K-means y KNN) mostraron altos niveles de asertividad. Para ambos algoritmos, se utilizaron 100 imágenes como conjunto de entrenamiento (25 imágenes de cada objeto). Únicamente se utilizó un objeto de cada tipo, es decir, una tuerca, una arandela, un clavo y un tornillo, para tomar las fotografías de entrenamiento. Por otro lado, cada imagen

presentaba orientaciones y niveles de iluminación ligeramente diferentes, lo cual sirvió para aumentar la robustez de los algoritmos.

La etapa de selección de características fue crucial a la hora de predecir correctamente los objetos. Al principio se utilizaron, como prueba inicial, todas las características calculadas. Con esa configuración se halló una precisión media de alrededor del 24%, lo cual no era aceptable. Al seleccionar cuidadosamente las características, la precisión aumentó al 100%.

Los resultados obtenidos a partir de los algoritmos de planificación y de búsqueda de rutas fueron aceptables desde un primer momento, y su implementación no provocó ningún problema en particular.

Al probar los demás algoritmos de búsqueda implementados, se encontró que, en general, el algoritmo A* modificado presenta ligeras ventajas de eficiencia. Este algoritmo generalmente requiere visitar menos cantidad de casillas hasta llegar la ruta optima, aunque la ruta óptima hallada sea la misma que la que obtiene el algoritmo A*.

8. Conclusiones

En el presente trabajo se implementó una gran variedad de algoritmos. En mi opinión, esto resultó en una experiencia de aprendizaje muy enriquecedora. Además, me pareció muy interesante explorar alternativas a los algoritmos solicitados por la consigna, como es el caso de los algoritmos de búsqueda primero en anchura y el algoritmo A* modificado.

Estoy muy satisfecho con el desarrollo de un algoritmo de generación automática de laberintos, ya que es la primera vez que realizo algo similar. El desarrollo de la interfaz gráfica también fue una dificultad adicional al proyecto.

En cuanto a la eficiencia de los algoritmos implementados, creo que no existe diferencia en cuanto a la utilización de Kmeans y KNN. Ambos logran predecir correctamente el objeto presente en la imagen cerca del 100% de las veces. Por otro lado, creo que el algoritmo A* modificado presenta ligeras ventajas de rendimiento con respecto al algoritmo clásico. Las rutas halladas muchas veces son las mismas, pero el algoritmo modificado las encuentra mucho más rápido.

Como trabajo futuro, se podrían incluir en el conjunto de entrenamiento, imágenes de diferentes objetos dentro de cada clase, ya que hasta ahora siempre se trabajó con el mismo tornillo, clavo, arandela y tuerca. Esto podría permitir la identificación más general del tipo de objeto en cuestión y aumentaría aún más la robustez de los algoritmos.

También se podría agregar alguna cámara conectada para ir tomando imágenes a tiempo real y ver el desempeño de los códigos en una situación mucho más profesional.

9. Referencias bibliográficas

[1] Gabrovšek, P. (2019). Analysis of maze generating algorithms. IPSI Transactions on Internet Research, 15(1), 23-30.

[2] Zhang Renping, Zhou Qingzhong (2009). Updated A* Algorithm and its Application, Computer Systems & Applications, 98-100.

[3] Ade Candra (2020). Application of A-Star Algorithm on Pathfinding Game, 5th International Conference on Computing and Applied Informatics.

[4] Xiang Liu, Daoxiong Gong (2011). A Comparative Study of A-star Algorithms for Search and rescue in Perfect Maze

[5] Stuart Russell; Peter Norvig (2010). Artificial Intelligence: A Modern Approach (3 edición). Prentice Hall.

[6] IBM. (2024). Documentación sobre K-Means Clustering. Recuperado de <https://www.ibm.com/mx-es/topics/k-means-clustering>.

[7] IBM. (2024). Documentación sobre KNN. Recuperado de <https://www.ibm.com/mx-es/topics/knn>.

10. Anexo: Código desarrollado

10.1. Clase base para los algoritmos de identificación de objetos

```
class ImageRecognitionAlgorithm:
    # Estos valores fueron obtenidos del Standard scaler después de
    # aplicarlo a los training instances (NO MODIFICAR)
    features_mean_ = np.array([0.38477562, 0.5522563, 0.55143874])
    features_var_ = np.array([0.0844822, 0.13646293, 0.13703266])

    category_mapping = {0: 'arandela', 1: 'clavo', 2: 'tornillo', 3: 'tuerca'}

    @staticmethod
    def preprocess_image(img):
        print(f'Original shape: {img.shape}')
        mean_color = img.mean(axis=(0, 1))
        print(f'Mean color (RGB): {mean_color}')

        # Threshold para la detección del fondo
        threshold = 85

        # Máscara donde los pixeles cerca del color del fondo se vuelven negros
        mask = np.linalg.norm(img - mean_color, axis=-1) < threshold
        img[mask] = [0, 0, 0] # Convert background to black

        # Convertir a escala de grises
        # Reducimos el número de canales de 3 a 1
        gray_img = color.rgb2gray(img)

        if gray_img.shape[1] > gray_img.shape[0]:
            gray_img = np.rot90(gray_img) # Rotate 90 degrees to make it portrait

        # Apply a median filter
        img_filtered = filters.rank.median(gray_img, disk(5))

        # Apply Li's threshold
        local_li = threshold_li(img_filtered)
        thresh_image = (img_filtered >= local_li).astype(np.uint8) * 255

        # Invert to obtain black background if needed
        if np.mean(thresh_image) > 127:
            thresh_image = cv2.bitwise_not(thresh_image)
            print(f"Inverting colors...")

        # Closing Filtering
        closed_image = binary_closing(thresh_image, disk(10))

        return gray_img, img_filtered, thresh_image, closed_image

    @staticmethod
    def search_contours(image, min_length=100):
        contours, _ = cv2.findContours(image, cv2.RETR_EXTERNAL, cv2.CHAIN_APPROX_SIMPLE)
        if not contours:
            return None # None si ningún contorno
        return max(contours, key=cv2.contourArea) # Devuelve el contorno mas grande

        #return [cnt for cnt in contours if cv2.arcLength(cnt, closed=True) >= min_length]

    @staticmethod
    def crop_object(image, contours):
        all_points = np.vstack(contours)
        cv2_contours = np.array(all_points, dtype=np.int32)

        # Calcula el convex_hull de los contornos de cada imagen
        convex_hull = cv2.convexHull(cv2_contours)

        # Get bounding rectangle around the convex hull
        x, y, w, h = cv2.boundingRect(convex_hull)

        # Crop the image using the bounding rectangle
        cropped = image[y:y + h, x:x + w]

        # Ajuste de coordenadas de los contornos y convex_hull
        # debido al recorte de la imagen
        adjusted_contours = [cnt - np.array([x, y]) for cnt in cv2_contours]
        adjusted_hull = convex_hull - np.array([x, y])
```



```

        return cropped, adjusted_contours, adjusted_hull

    @staticmethod
    def compute_features(image, contours, hull):
        # Nos aseguramos que los contornos y el convex hull son np.array
        contours = np.array(contours, dtype=np.float32) if not isinstance(contours, np.ndarray) else contours
        hull = np.array(hull, dtype=np.float32) if not isinstance(hull, np.ndarray) else hull

        # Cálculo de los momentos de Hu (convex_hull)
        moments = cv2.moments(hull)
        hu_moments = cv2.HuMoments(moments).flatten()

        # Cálculo del área y perímetro de los contornos de la imagen
        area = cv2.contourArea(contours)
        perimeter = cv2.arcLength(contours, True)

        # Circularidad
        circularity = (4 * np.pi * area) / (perimeter ** 2) if perimeter != 0 else 0

        # Características geométricas del convex hull
        hull_area = cv2.contourArea(hull)
        hull_perimeter = cv2.arcLength(hull, closed=True)

        solidity = area / hull_area if hull_area != 0 else 0
        convexity = hull_perimeter / perimeter if perimeter != 0 else 0

        # Cálculo del menor círculo que encierra al convex hull
        (x, y), radius = cv2.minEnclosingCircle(hull)
        circle_area = math.pi * (radius ** 2)

        area_ratio = hull_area / circle_area if circle_area != 0 else 0

        # Ellipse fitting
        if len(hull) >= 5: # cv2.fitEllipse requires at least 5 points
            ellipse = cv2.fitEllipse(hull)
            major_axis = max(ellipse[1])
            minor_axis = min(ellipse[1])
            axis_aspect_ratio = minor_axis / major_axis if major_axis != 0 else 0
            eccentricity = np.sqrt(1 - (minor_axis ** 2 / major_axis ** 2)) if major_axis != 0 else 0
        else:
            major_axis, minor_axis, axis_aspect_ratio, eccentricity = 0, 0, 0, 0

        # Perímetro / Área (No muy útil)
        per_area_aspect_ratio = perimeter / hull_area if hull_area != 0 else 0

        # Solo conservamos el primer momento de Hu, el circleAreaRatio y el eccentricity
        features = np.append(hu_moments[0], np.array([area_ratio, eccentricity]))
        return features

    def scale_features(self, features):
        features = features.reshape(-1) # Ensure (3,) shape
        mean = self.features_mean_.reshape(-1) # Ensure (3,) shape
        var = self.features_var_.reshape(-1) # Ensure (3,) shape

        if features.shape[0] != mean.shape[0]:
            raise ValueError(f"Feature dimension mismatch: {features.shape[0]} vs {mean.shape[0]}")

        predict_features_scaled = (features - mean) / np.sqrt(var)
        return predict_features_scaled # Ensuring (3,) shape

    @staticmethod
    def draw_convex_hull(cropped_image, adjusted_hull):
        # Convert to BGR if the image is grayscale
        if len(cropped_image.shape) == 2: # Grayscale
            cropped_image = cv2.cvtColor(cropped_image, cv2.COLOR_GRAY2BGR)

        # Si existe el convex_hull, lo añadimos a la imagen
        if len(adjusted_hull) > 0:
            hull_array = np.array([adjusted_hull], dtype=np.int32)
            cv2.polylines(cropped_image, hull_array, isClosed=True, color=(255, 0, 0), thickness=4)

        return cropped_image # Returns the numpy array with the red convex hull

    def plot_3d_comparison(self, predict_features_scaled, train_features_scaled, train_labels, filename):

```

```

# Create a 3D plot
fig = plt.figure(figsize=(14, 10))
ax = fig.add_subplot(111, projection='3d')
ax.view_init(elev=35, azim=65)

# Plot the training instances
scatter = ax.scatter(train_features_scaled[:, 0], train_features_scaled[:, 1],
train_features_scaled[:, 2],
                      c=train_labels, cmap='viridis', label='Datos de entrenamiento', alpha=0.6)

# Plot the predicted instance
ax.scatter(predict_features_scaled[0], predict_features_scaled[1], predict_features_scaled[2],
           c='red', marker='X', s=200, label='Punto de datos de entrada')

# Create a custom legend for the training data colors
unique_labels = np.unique(train_labels)
cmap = ListedColormap(plt.cm.viridis(np.linspace(0, 1, len(unique_labels))))
legend_elements = [Patch(facecolor=cmap(i), label=f'Clase {self.category_mapping[int(label)]}') for
i, label in enumerate(unique_labels)]

# Add the predicted feature to the legend
legend_elements.append(Patch(facecolor='red', label='Punto de datos de entrada'))
ax.legend(handles=legend_elements, title="Legend", bbox_to_anchor=(1.15, 1), loc='upper left')

plt.title('Gráfico 3D de comparación', pad=20)

plt.subplots_adjust(left=0.1, right=0.8, bottom=0.1, top=0.9)
ax.set_xlabel('Característica 1 (Scaled Hu Moment 1)', labelpad=15, )
ax.set_ylabel('Característica 2 (Scaled Ratio de Área circular)', labelpad=15)
ax.set_zlabel('Característica 3 (Scaled Excentricidad)', labelpad=15)
# Save the plot
plt.savefig(filename, format='png', dpi=300, bbox_inches='tight', pad_inches=1)
plt.close()

```

10.2. Algoritmo KNN

```

class KNN(ImageRecognitionAlgorithm):
    def __init__(self, k, train_filename, predict_image):
        self.k = k
        self.predict_image = predict_image

    # Carga las características de entrenamiento a partir de un archivo CSV.
    self.train_features = np.loadtxt(train_filename, delimiter=',', skiprows=1) # Skip header
    self.X = self.train_features[:, :-1]
    self.y = self.train_features[:, -1]

    def launch_knn(self):
        # Procesa la imagen original
        gray_img, img_filtered, thresh_image, closed_image = self.preprocess_image(self.predict_image)

        # Nos aseguramos que la imagen esté en el formato correcto (uint8)
        if closed_image.dtype != np.bool_:
            closed_image = closed_image.astype(np.uint8) * 255

        # Búsqueda de contornos
        contours = self.search_contours(closed_image, min_length=100)

        # Recorta la imagen alrededor del contorno principal (convex_hull)
        cropped_image, adjusted_contours, adjusted_hull = self.crop_object(closed_image, contours)

        # Construye la imagen que luego se muestra en la interfaz de usuario
        hull_image = self.draw_convex_hull(cropped_image, adjusted_hull)

        # Cálculo de características
        predict_features = self.compute_features(cropped_image, adjusted_contours, adjusted_hull)

        # Procesamiento de características (cambio de escala)
        predict_features_scaled = self.scale_features(predict_features)

        # Algoritmo KNN
        if self.X.shape[1] != predict_features_scaled.shape[0]:
            raise f"Invalid input shape. It should be {self.X.shape} but it is {predict_features_scaled.shape}."

```

```

if self.k <= len(np.unique(self.y)):
    warnings.warn('K is set to a value less than total voting groups.')

distances = []
for index, instance in enumerate(self.X):
    euclidean_distance = np.linalg.norm(instance - predict_features_scaled)
    distances.append([euclidean_distance, self.y[index]])

votes = [i[1] for i in sorted(distances)[:self.k]]
vote_result = Counter(votes).most_common(1)[0]

vote_result_categorical = self.category_mapping[int(vote_result[0])] # Devuelve el nombre de la
clase ('arandela' por ejemplo)
confidence = vote_result[1] / self.k

self.plot_3d_comparison(predict_features_scaled, self.X, self.y, '3d_plot_comparison.png')

return vote_result_categorical, confidence, hull_image, predict_features, predict_features_scaled

```

10.3. Algoritmo KMeans

```

class KMeans(ImageRecognitionAlgorithm):
    def __init__(self, train_filename, predict_image, k=4, max_iters=100, tol=1e-4, n_init = 10):
        self.predict_image = predict_image

        # Carga los datos de entrenamiento a partir de un archivo CSV
        self.train_features = np.loadtxt(train_filename, delimiter=',', skiprows=1) # Skip header
        self.X = self.train_features[:, :-1]
        self.y = self.train_features[:, -1]

        self.k = k
        self.max_iters = max_iters
        self.tol = tol
        self.n_init = n_init

    @staticmethod
    def initialize_centroids(X, k):
        """Elige k centroides aleatoriamente."""
        indices = np.random.choice(X.shape[0], k, replace=False)
        return X[indices]

    @staticmethod
    def kmeans_plusplus_initialization(X, k):
        """
        Inicialización Kmeans++ para seleccionar los k centroides iniciales.
        """
        n_samples, n_features = X.shape

        # Paso 1 : Elegir aleatoriamente el primer centroide
        centroids = [X[np.random.choice(n_samples)]]

        for _ in range(1, k):
            # Paso 2: Calcular la distancia al centroide más cercano
            distances = np.array([min([np.linalg.norm(x - c) ** 2 for c in centroids]) for x in X])

            # Paso 3: Selecciona el siguiente centroide con probabilidad proporcional a la distancia
            probabilities = distances / distances.sum()
            next_centroid_idx = np.random.choice(n_samples, p=probabilities)
            centroids.append(X[next_centroid_idx])

        return np.array(centroids)

    @staticmethod
    def assign_clusters(X, centroids):
        """Asigna cada punto al centroide más cercano."""
        distances = np.linalg.norm(X[:, np.newaxis] - centroids, axis=2)
        return np.argmin(distances, axis=1)

    @staticmethod
    def update_centroids(X, labels, k):
        """Calcula los nuevos centroides como la media de los puntos en cada cluster."""
        return np.array([X[labels == i].mean(axis=0) for i in range(k)])

```

```

def predict(self, predict, centroids):
    return self.assign_clusters(predict, centroids)

@staticmethod
def cluster_to_label_mapping(y_true, y_pred):
    """Finds the most frequent true label for each cluster."""
    unique_clusters = np.unique(y_pred)
    cluster_mapping = {}

    for cluster in unique_clusters:
        mask = y_pred == cluster
        most_common_label = mode(y_true[mask]).mode
        cluster_mapping[int(cluster)] = int(most_common_label)

    return cluster_mapping

@staticmethod
def clustering_accuracy(y_true, y_pred, cluster_mapping):
    """Calcula la precision del clustering en base al mapeo."""
    correct_count = np.sum([cluster_mapping[label] == y_true[i] for i, label in enumerate(y_pred)])
    total_count = len(y_true)
    return correct_count / total_count

def plot_3d_comparison(self, predict_features_scaled, train_features_scaled, train_labels, filename):

    fig = plt.figure(figsize=(14, 10))
    ax = fig.add_subplot(111, projection='3d')
    ax.view_init(elev=35, azim=65)

    # Datos de entrenamiento
    scatter = ax.scatter(train_features_scaled[:, 0], train_features_scaled[:, 1],
train_features_scaled[:, 2],
                        c=train_labels, cmap='viridis', label='Datos de entrenamiento', alpha=0.6)

    # Dato de entrada
    ax.scatter(predict_features_scaled[0], predict_features_scaled[1], predict_features_scaled[2],
              c='red', marker='X', s=200, label='Punto de datos de entrada')

    unique_labels = np.unique(train_labels)
    cmap = ListedColormap(plt.cm.viridis(np.linspace(0, 1, len(unique_labels))))
    legend_elements = [Patch(facecolor=cmap(i), label=f'Grupo {int(label)}') for i, label in
enumerate(unique_labels)]

    legend_elements.append(Patch(facecolor='red', label='Punto de datos de entrada'))

    ax.legend(handles=legend_elements, title="Legend", bbox_to_anchor=(1.15, 1), loc='upper left')

    plt.title('Gráfico 3D de comparación', pad=20)

    plt.subplots_adjust(left=0.1, right=0.8, bottom=0.1, top=0.9)
    ax.set_xlabel('Característica 1 (Scaled Hu Moment 1)', labelpad=15)
    ax.set_ylabel('Característica 2 (Scaled Ratio de área circular)', labelpad=15)
    ax.set_zlabel('Característica 3 (Scaled Excentricidad)', labelpad=15)

    plt.savefig(filename, format='png', dpi=300, bbox_inches='tight', pad_inches=1)
    plt.close()

def launch_kmeans(self):
    # Procesamiento de la imagen original
    gray_img, img_filtered, thresh_image, closed_image = self.preprocess_image(self.predict_image)

    # Nos aseguramos que la imagen esté en el formato correcto (uint8)
    if closed_image.dtype == np.bool_:
        closed_image = closed_image.astype(np.uint8) * 255 # Convert boolean to uint8 (0 or 255)

    # Búsqueda de contornos
    contours = self.search_contours(closed_image, min_length=100)

    # Recorta la imagen alrededor del contorno principal
    cropped_image, adjusted_contours, adjusted_hull = self.crop_object(closed_image, contours)

    # Obtiene la imagen que luego se muestra en la interfaz de usuario
    hull_image = self.draw_convex_hull(cropped_image, adjusted_hull)

    # Cálculo de características
    predict_features = self.compute_features(cropped_image, adjusted_contours, adjusted_hull)

```

```

# Procesamiento de características (cambio de escala)
predict_features_scaled = self.scale_features(predict_features)

# Algoritmo Kmeans
best_inertia = np.inf
best_centroids = None
best_labels = None

for _ in range(self.n_init):
    centroids = self.kmeans_plusplus_initialization(self.X, self.k)

    for _ in range(self.max_iters):
        labels = self.assign_clusters(self.X, centroids)
        new_centroids = self.update_centroids(self.X, labels, self.k)

        # Convergencia ?
        if np.linalg.norm(new_centroids - centroids) < self.tol:
            break

    centroids = new_centroids

# Cálculo de la inercia del clustering resultante
inertia = np.sum([np.linalg.norm(self.X[labels == j] - centroids[j]) ** 2 for j in
range(self.k)])

# Actualiza el mejor resultado
if inertia < best_inertia:
    best_inertia = inertia
    best_centroids = centroids
    best_labels = labels

print(f"Centroides encontrados: {best_centroids}")

# Cluster to Label mapping
mapping = self.cluster_to_label_mapping(np.array(self.y), np.array(best_labels))
print("Mapa Cluster->Clase:", mapping)

accuracy = self.clustering_accuracy(self.y, best_labels, mapping)
print(f"Precisión del agrupamiento: {accuracy:.2%}")

# Predice la imagen
predicted_group = self.predict(predict_features_scaled.reshape(1, -1), best_centroids)[0]
predicted_label_int = mapping[predicted_group]
predicted_label = self.category_mapping[predicted_label_int]
print(f"Grupo predicho: {predicted_group} - Clase mapeada: {predicted_label_int} - Clase:
{predicted_label}")

self.plot_3d_comparison(predict_features_scaled, self.X, best_labels, '3d_plot_comparison.png')

return predicted_group, predicted_label, accuracy, hull_image, predict_features,
predict_features_scaled

```

10.4. Clase base para los algoritmos de búsqueda

```

class Solver(QObject):
    solve_completed = pyqtSignal()

    def __init__(self, maze):
        super().__init__()
        self.maze = maze
        self.openSet = []
        self.cameFrom = {}
        self.g_score = {}
        self.f_score = {}
        self.controller = None # To be set by the controller
        self.timer = QTimer()
        self.timer.timeout.connect(self.step)
        self.endCell = None
        self.visited_count = 0
        self.start_time = 0
        self.end_time = 0

    def heuristic(self, current, target):

```

```

        raise NotImplementedError("Subclasses should implement heuristic.")

    def eval_function(self, values):
        raise NotImplementedError("Subclasses should implement eval_function.")

    def get_neighbors(self, node):
        neighbors = []
        rowID, columnID = node
        if self.maze.isWithinBounds(rowID + 1, columnID) and self.maze.isWalkable(rowID + 1, columnID):
            neighbors.append((rowID + 1, columnID))
        if self.maze.isWithinBounds(rowID - 1, columnID) and self.maze.isWalkable(rowID - 1, columnID):
            neighbors.append((rowID - 1, columnID))
        if self.maze.isWithinBounds(rowID, columnID + 1) and self.maze.isWalkable(rowID, columnID + 1):
            neighbors.append((rowID, columnID + 1))
        if self.maze.isWithinBounds(rowID, columnID - 1) and self.maze.isWalkable(rowID, columnID - 1):
            neighbors.append((rowID, columnID - 1))
        return neighbors

    def reconstruct_path(self, current):
        path = []
        while current in self.cameFrom:
            path.append(current)
            current = self.cameFrom[current]
        path.reverse()
        return path

    def solve(self):
        raise NotImplementedError("Subclasses should implement solve.")

    def step(self):
        raise NotImplementedError("Subclasses should implement step.")

```

10.5. Algoritmo A*

```

class AStarSolver(Solver):
    def __init__(self, maze):
        super().__init__(maze)

    def heuristic(self, current, target):
        (x1, y1), (x2, y2) = current, target
        #return np.sqrt((x1-x2)**2 + (y1-y2)**2) # Euclidean Distance
        return abs(x1 - x2) + abs(y1 - y2) # Manhattan Distance

    def eval_function(self, values):
        if isinstance(values, list):
            return sum(values)
        elif isinstance(values, int):
            return values
        else:
            raise TypeError("Score values should be either a list or an integer")

    def solve(self):
        self.visited_count = 0
        self.start_time = datetime.now()
        start = self.maze.startCell
        self.endCell = self.maze.endCell

        if not start or not self.endCell:
            return None

        self.openSet = []
        # Push start node into the priority queue
        heapq.heappush(self.openSet, (0, start))
        self.cameFrom = {}

        self.g_score = {start: 0} # Traveled distance from starting node to the starting node is zero
        self.f_score = {start: self.eval_function([self.g_score[start], self.heuristic(start,
self.endCell)])}

        self.maze.clearSetNodes()
        self.timer.start(10) # Adjust the interval as needed

    def step(self):

```

```

if not self.openSet:
    self.timer.stop()
    return
# Pop first element of the queue (node with minimum value of f_score)
_, current = heapq.heappop(self.openSet)

# Check if we've reached the goal
if current == self.endCell:
    self.timer.stop()
    self.end_time = datetime.now()
    path = self.reconstruct_path(current)
    self.controller.maze_widget.path = path
    self.controller.maze_widget.path_stats["N_Visited"] = self.visited_count
    self.controller.maze_widget.path_stats["Time_Spent"] = self.end_time - self.start_time
    self.controller.maze_widget.path_stats["Route_Cost"] = self.g_score.get(current, float('inf'))
    self.solve_completed.emit()
    self.controller.maze_widget.update()
    return

self.visited_count += 1
self.maze.addClosedSetNode(*current)
self.controller.maze_widget.update()
for neighbor in self.get_neighbors(current):
    if neighbor in self.maze.closedSetNodes:
        continue
    # Get cost from the start node to the current node (defaulting to infinity if it's not found),
    # adds the cost of moving from the current node to the neighbor which is assumed to be 1
    tentative_g_score = self.g_score.get(current, float('inf')) + 1

    if tentative_g_score < self.g_score.get(neighbor, float('inf')):
        self.cameFrom[neighbor] = current
        self.g_score[neighbor] = tentative_g_score
        self.f_score[neighbor] = self.eval_function(
            [self.g_score[neighbor], self.heuristic(neighbor, self.endCell)])
        # Adds the neighbor to the priority queue (open set) with its f_score as the priority.
        # The priority queue ensures nodes with the lowest f_score are explored first.
        heapq.heappush(self.openSet, (self.f_score[neighbor], neighbor))
        self.maze.addOpenSetNode(*neighbor)
self.controller.maze_widget.update()

```

10.6. Algoritmo A* modificado

```

class ModifiedAStarSolver(Solver):
    def __init__(self, maze):
        super().__init__(maze)

    def heuristic(self, current, target):
        (x1, y1), (x2, y2) = current, target
        return abs(x1 - x2) + abs(y1 - y2)

    def eval_function(self, values):
        if isinstance(values, list):
            return sum(values)
        elif isinstance(values, int):
            return values
        else:
            raise TypeError("Score values should be either a list or an integer")

    def solve(self):
        self.visited_count = 0
        self.start_time = datetime.now()
        start = self.maze.startCell
        self.endCell = self.maze.endCell

        if not start or not self.endCell:
            return None

        self.openSet = []
        # Push start node into the priority queue
        heapq.heappush(self.openSet, (0, start))
        self.cameFrom = {}

        self.g_score = {start: 0} # Traveled distance from starting node to the starting node is zero

```

```

        self.f_score = {start: self.eval_function([self.g_score[start], self.heuristic(start,
self.endCell)])}

        self.maze.clearSetNodes()
        self.timer.start(10) # Adjust the interval as needed

    def step(self):
        if not self.openSet:
            self.timer.stop()
            return
        # Pop first element of the queue (node with minimum value of f_score)
        _, current = heapq.heappop(self.openSet)

        if current == self.endCell:
            self.timer.stop()
            self.end_time = datetime.now()
            path = self.reconstruct_path(current)
            self.controller.maze_widget.path = path
            self.controller.maze_widget.path_stats["N_Visited"] = self.visited_count
            self.controller.maze_widget.path_stats["Time_Spent"] = self.end_time - self.start_time
            self.controller.maze_widget.path_stats["Route_Cost"] = self.g_score.get(current, float('inf'))
            self.solve_completed.emit()
            self.controller.maze_widget.update()
            return

        self.visited_count += 1
        self.maze.addClosedSetNode(*current)
        self.controller.maze_widget.update()

        for neighbor in self.get_neighbors(current):
            if neighbor in self.maze.closedSetNodes:
                continue
            # Get cost from the start node to the current node (defaulting to infinity if it's not found),
            # adds the cost of moving from the current node to the neighbor which is assumed to be 1
            tentative_g_score = self.g_score.get(current, float('inf')) + 1

            if tentative_g_score < self.g_score.get(neighbor, float('inf')):
                self.cameFrom[neighbor] = current
                self.g_score[neighbor] = tentative_g_score
                self.f_score[neighbor] = self.eval_function(
                    [self.g_score[neighbor], self.heuristic(neighbor, self.endCell),
                    self.heuristic(current, self.endCell)])
                # Adds the neighbor to the priority queue (open set) with its f_score as the priority.
                # The priority queue ensures nodes with the lowest f_score are explored first.
                heapq.heappush(self.openSet, (self.f_score[neighbor], neighbor))
                self.maze.addOpenSetNode(*neighbor)
                self.controller.maze_widget.update()

```

10.7. Algoritmo BPA

```

class BPASolver(Solver):
    def __init__(self, maze):
        super().__init__(maze)

    def heuristic(self, current, target):
        # Dijkstra's algorithm does not use a heuristic
        return 0

    def eval_function(self, values):
        # Dijkstra's algorithm uses only the g_score
        if isinstance(values, list):
            return values[0]
        elif isinstance(values, int):
            return values
        else:
            raise TypeError("Score values should be either a list or an integer")

    def solve(self):
        self.visited_count = 0
        self.start_time = datetime.now()
        start = self.maze.startCell
        self.endCell = self.maze.endCell

        if not start or not self.endCell:
            return None

        self.openSet = []

```



```

# Push start node into the priority queue
heapq.heappush(self.openSet, (0, start))
self.cameFrom = {}

self.g_score = {start: 0} # Traveled distance from starting node to the starting node is zero
self.f_score = {start: self.eval_function([self.g_score[start]])}

self.maze.clearSetNodes()
self.timer.start(10) # Adjust the interval as needed

def step(self):
    if not self.openSet:
        self.timer.stop()
        return
    # Pop first element of the queue (node with minimum value of f_score)
    _, current = heapq.heappop(self.openSet)

    if current == self.endCell:
        self.timer.stop()
        self.end_time = datetime.now()
        path = self.reconstruct_path(current)
        self.controller.maze_widget.path = path
        self.controller.maze_widget.path_stats["N_Visited"] = self.visited_count
        self.controller.maze_widget.path_stats["Time_Spent"] = self.end_time - self.start_time
        self.controller.maze_widget.path_stats["Route_Cost"] = self.g_score.get(current, float('inf'))
        self.solve_completed.emit()
        self.controller.maze_widget.update()
        return

    self.visited_count += 1
    self.maze.addClosedSetNode(*current)
    self.controller.maze_widget.update()

    for neighbor in self.get_neighbors(current):
        if neighbor in self.maze.closedSetNodes:
            continue
        # Get cost from the start node to the current node (defaulting to infinity if it's not found),
        # adds the cost of moving from the current node to the neighbor which is assumed to be 1
        tentative_g_score = self.g_score.get(current, float('inf')) + 1

        if tentative_g_score < self.g_score.get(neighbor, float('inf')):
            self.cameFrom[neighbor] = current
            self.g_score[neighbor] = tentative_g_score
            self.f_score[neighbor] = self.eval_function([self.g_score[neighbor]])
            # Adds the neighbor to the priority queue (open set) with its f_score as the priority.
            # The priority queue ensures nodes with the lowest f_score are explored first.
            heapq.heappush(self.openSet, (self.f_score[neighbor], neighbor))
            self.maze.addOpenSetNode(*neighbor)
            self.controller.maze_widget.update()

```

10.8. Clase correspondiente al Laberinto

```

class Maze:
    def __init__(self, rows, columns):
        self.nbRows = rows
        self.nbColumns = columns
        self.grid = np.zeros((rows, columns), dtype=int)
        self.startCell = None
        self.endCell = None
        self.openSetNodes = set()
        self.closedSetNodes = set()

    def setStartCell(self, rowID, columnID):
        self.startCell = (rowID, columnID)
        self.grid[rowID][columnID] = 2 # Start point

    def setEndCell(self, rowID, columnID):
        self.endCell = (rowID, columnID)
        self.grid[rowID][columnID] = 3 # End point

    def setWallCell(self, rowID, columnID):
        self.grid[rowID][columnID] = 1 # Wall

    def addOpenSetNode(self, rowID, columnID):
        self.openSetNodes.add((rowID, columnID))

    def addClosedSetNode(self, rowID, columnID):
        self.closedSetNodes.add((rowID, columnID))

    def clearSetNodes(self):
        self.openSetNodes.clear()

```

```

self.closedSetNodes.clear()

def isWithinBounds(self, rowID, columnID):
    return 0 <= rowID < self.nbRows and 0 <= columnID < self.nbColumns

def isWalkable(self, rowID, columnID):
    return self.grid[rowID][columnID] == 0 or self.grid[rowID][columnID] == 3

def __repr__(self):
    return str(self.grid)

def clearMaze(self):
    self.startCell = None
    self.endCell = None
    self.clearSetNodes()
    # Initially all cells are considered as walls
    self.grid = np.zeros((self.nbRows, self.nbColumns), dtype=int)

def generateMaze(self):
    in_cells = set()
    frontier_cells = set()

    self.clearMaze()

    # Initially all cells are considered as walls
    self.grid = np.ones((self.nbRows, self.nbColumns), dtype=int)

    start_row, start_col = random.randrange(0, self.nbRows), random.randrange(0, self.nbColumns)
    self.grid[start_row][start_col] = 0
    in_cells.add((start_row, start_col))

    directions = [(0, 2), (2, 0), (0, -2), (-2, 0)]

    for direction in directions:
        new_row, new_col = start_row + direction[0], start_col + direction[1]
        if self.isWithinBounds(new_row, new_col) and self.grid[new_row][new_col] == 1:
            frontier_cells.add((new_row, new_col))

    while len(frontier_cells) > 0:
        cf = random.choice(list(frontier_cells))
        cf_row, cf_col = cf
        random.shuffle(directions)
        found_in_cell = False
        for direction in directions:
            in_row, in_col = cf_row + direction[0], cf_col + direction[1]
            if (in_row, in_col) in in_cells and not found_in_cell:
                found_in_cell = True
                self.grid[cf_row][cf_col] = 0
                self.grid[cf_row + direction[0] // 2][cf_col + direction[1] // 2] = 0
                in_cells.add(cf)
                frontier_cells.remove(cf)

        elif (in_row, in_col) not in in_cells:
            if self.isWithinBounds(in_row, in_col) and self.grid[in_row][in_col] == 1:
                frontier_cells.add((in_row, in_col))

```

10.9. Clase al controlador de la interfaz

Este objeto se encarga de conectar la parte visual con los algoritmos desarrollados. Aquí se ejecutan también la gran parte de las funciones vinculadas a la planificación del reordenamiento de las cajas.

```

class Controller:
    def __init__(self, maze):
        self.maze = maze
        self.solver = None
        self.maze_widget = None # Initialize it later

    # PDDL parameters
    self.problem_name = "CAJAS-4"
    self.domain_name = "CAJAS_domain"
    self.objects = "ARANDELA TUERCA CLAVO TORNILLO"

```

```

        self.problem_file = "CAJAS_problem.pddl"

    def set_solver(self, solver):
        self.solver = solver
        self.solver.controller = self # Link solver to controller

    def solve_maze(self):
        if self.solver:
            self.solver.solve()

    def generate_maze(self):
        if self.maze:
            self.maze.generateMaze()

    def clear_maze(self):
        if self.maze:
            self.maze.clearMaze()

    """
    STRIPS Box Order Functionalities
    """

    @staticmethod
    def get_rectangles_order(rectangles):
        rectangles_order = [(index, rectangle["label"].upper()) for index, rectangle in
enumerate(rectangles)]
        return rectangles_order

    def write_pddl_problem_file(self, rectangles_init, rectangles_goal):
        init_order = self.get_rectangles_order(rectangles_init)
        goal_order = self.get_rectangles_order(rectangles_goal)

        init_string = f"""(MANOLIBRE)
(LIBRE {init_order[0][1]})
(ENMESA {init_order[3][1]})
(SOBRE {init_order[0][1]} {init_order[1][1]})
(SOBRE {init_order[1][1]} {init_order[2][1]})
(SOBRE {init_order[2][1]} {init_order[3][1]})
"""

        goal_string = f"""(MANOLIBRE)
(LIBRE {goal_order[0][1]})
(ENMESA {goal_order[3][1]})
(SOBRE {goal_order[0][1]} {goal_order[1][1]})
(SOBRE {goal_order[1][1]} {goal_order[2][1]})
(SOBRE {goal_order[2][1]} {goal_order[3][1]})
"""

        #print(init_string)
        #print(goal_string)

        problem_content = f"""(define
(problem {self.problem_name})
(:domain {self.domain_name})
(:objects {"".join(self.objects)})
(:init {"".join(init_string)})
(:goal (and {"".join(goal_string)}))
)
"""

        #print((problem_content))
        try:
            with open(self.problem_file, 'w') as file:
                file.write(problem_content)
                print(f"Problem file '{self.problem_file}' created successfully.")
                return self.problem_file
        except:
            return None

    def search_ppdl_solution(self, rectangles_init, rectangles_goal):
        filename = self.write_pddl_problem_file(rectangles_init, rectangles_goal)
        if filename is None:
            raise "Couldn't create PDDL problem file."

    def run_planner(problem_file, domain_file, search_algorithm="astar(blind())", ):

```

```

    """
    Runs the Fast Downward planner with the specified search algorithm.
    """
    root = 'downward'
    try:
        # Command to run Fast Downward
        command = [
            "python",
            f"{root}/fast-downward.py", # Path to the Fast Downward script
            domain_file,
            problem_file,
            "--search", search_algorithm
        ]

        # Execute the command and capture the output
        result = subprocess.run(command, capture_output=True, text=True, check=True)

        # Print the result's stdout (planner output)
        print("Planner output:")
        print(result.stdout)
        return result.stdout
    except subprocess.CalledProcessError as e:
        print(f"An error occurred: {e.stderr}")
    except FileNotFoundError:
        print(
            "Fast Downward executable not found. Make sure you've built Fast Downward and set the
correct path.")

    domain_file = self.domain_name + ".pddl"
    print(domain_file)
    planning_result = run_planner(problem_file=filename, search_algorithm="astar(lmcut())",
domain_file=domain_file)
    return self.extract_actions(planning_result)

    @staticmethod
    def extract_actions(log):
        # Define action keywords to look for
        action_keywords = ["desapilar", "bajar", "agarrar", "apilar"]
        actions = []

        # Split the log into lines and check each line
        for line in log.splitlines():
            if any(line.startswith(keyword) for keyword in action_keywords):
                actions.append(line.strip()) # Add the action to the list

        return actions

    """Object Recognition Functionalities"""
    def launch_algorithm(self, algorithm, image):
        if algorithm == "K-Means":
            # Calls KMeans
            kmeans = KMeans(train_filename='train_features.csv', predict_image=image)
            vote_group, vote_result, confidence, hull_image, features, scaled_features =
kmeans.launch_kmeans()

            features_names = ['Hu_Moment_1', 'Circle_Area_Ratio', 'Eccentricity']
            features_dict = dict(zip(features_names, features))
            scaled_features_names = ['Scaled_Hu_Moment_1', 'Scaled_Circle_Area_Ratio',
'Scaled_Eccentricity']
            features_dict_scaled = dict(zip(scaled_features_names, scaled_features))

            return [vote_group, vote_result, confidence, hull_image, features_dict, features_dict_scaled]

        elif algorithm == "KNN":
            # Calls KNN
            knn = KNN(k=7, train_filename='train_features.csv', predict_image=image)
            vote_result, confidence, hull_image, features, scaled_features = knn.launch_knn()

            features_names = ['Hu_Moment_1', 'Circle_Area_Ratio', 'Eccentricity']
            features_dict = dict(zip(features_names, features))
            scaled_features_names = ['Scaled_Hu_Moment_1', 'Scaled_Circle_Area_Ratio',
'Scaled_Eccentricity']
            features_dict_scaled = dict(zip(scaled_features_names, scaled_features))

            return [vote_result, confidence, hull_image, features_dict, features_dict_scaled]

```

```

else:
    raise "Invalid algorithm."

```

10.10. Clase correspondiente a la ventana del laberinto

```

class MainWindow(QMainWindow):
    def __init__(self, controller):
        super().__init__()
        self.controller = controller

        self.setWindowTitle("Trabajo Final - Inteligencia Artificial 1 - CORAZZA")
        # Get screen resolution
        screen_resolution = QDesktopWidget().screenGeometry()
        width, height = int(screen_resolution.width()), int(screen_resolution.height())

        title_font = QFont("Arial", 15)
        normal_font = QFont("Arial", 10)

        self.setGeometry(0, 0, width, height)

        self.maze_widget = MazeWidget(self.controller.maze, self.controller)
        self.solve_button = QPushButton("Buscar ruta")
        self.solve_button.setFont(normal_font)
        self.solve_button.setStyleSheet("background-color: #CCD2CD;")
        self.solve_button.clicked.connect(self.controller.solve_maze)

        self.generate_button = QPushButton("Generar laberinto")
        self.generate_button.setFont(normal_font)
        self.generate_button.setStyleSheet("background-color: #CCD2CD;")
        self.generate_button.clicked.connect(self.generate_maze)

        self.clear_button = QPushButton("Borrar laberinto")
        self.clear_button.setFont(normal_font)
        self.clear_button.setStyleSheet("background-color: #CCD2CD;")
        self.clear_button.clicked.connect(self.clear_maze)

        self.title_label = QLabel("Transporte de cajas")
        self.title_label.setFont(title_font)
        self.title_label.setContentsMargins(0, 0, 0, 10)

        self.solver_label = QLabel("Algoritmo de búsqueda: ")
        self.solver_label.setFont(normal_font)
        self.solver_label.setContentsMargins(5, 5, 0, 0)

        self.astar_radio = QRadioButton("Algoritmo A*")
        self.astar_radio.setFont(normal_font)
        self.dijkstra_radio = QRadioButton("Búsqueda Primero en Anchura")
        self.dijkstra_radio.setFont(normal_font)
        self.modified_astar_radio = QRadioButton("Algoritmo A* modificado")
        self.modified_astar_radio.setFont(normal_font)

        self.astar_radio.setChecked(True)

        self.radio_group = QButtonGroup()
        self.radio_group.addButton(self.astar_radio)
        self.radio_group.addButton(self.dijkstra_radio)
        self.radio_group.addButton(self.modified_astar_radio)

        radio_layout = QVBoxLayout()
        radio_layout.addWidget(self.astar_radio)
        radio_layout.addWidget(self.dijkstra_radio)
        radio_layout.addWidget(self.modified_astar_radio)
        radio_layout.setContentsMargins(20, 10, 10, 30)

        left_layout = QVBoxLayout()
        left_layout.setContentsMargins(30, 0, 30, 0)
        left_layout.setAlignment(Qt.AlignTop)
        left_layout.addWidget(self.title_label, alignment=Qt.AlignCenter)
        left_layout.addWidget(self.solver_label)
        left_layout.addLayout(radio_layout)
        left_layout.addWidget(self.solve_button)
        left_layout.addWidget(self.generate_button)
        left_layout.addWidget(self.clear_button)

        main_layout = QHBoxLayout()
        main_layout.addLayout(left_layout, stretch=2)

        # Create a centered layout for the maze widget
        centered_layout = QVBoxLayout()
        centered_layout.addWidget(self.maze_widget, alignment=Qt.AlignCenter)
        main_layout.addLayout(centered_layout, stretch=4)

        # STRIPS Box Order Widget

```

```

container = QWidget()
container.setLayout(main_layout)

self.stacked_widget = QStackedWidget()
self.stacked_widget.addWidget(container)
self.box_order_window = BoxOrderWindow(self.controller)
self.stacked_widget.addWidget(self.box_order_window)
self.object_recognition_window = ObjectRecognitionWindow(controller)
self.stacked_widget.addWidget(self.object_recognition_window)

self.setCentralWidget(self.stacked_widget)
self.controller.set_solver(self.get_selected_solver())

self.setCentralWidget(self.stacked_widget)

self.controller.set_solver(self.get_selected_solver())

self.astar_radio.toggled.connect(self.on_solver_changed)
self.dijkstra_radio.toggled.connect(self.on_solver_changed)
self.modified_astar_radio.toggled.connect(self.on_solver_changed)

self.setStyleSheet("background-color: lightblue;")

# Add label to show path_stats
self.path_stats_label = QLabel()
self.path_stats_label.setContentsMargins(15, 10, 0, 0)
self.path_stats_label.setFont(normal_font)
self.path_stats_label.setStyleSheet("color: black; font-size: 20px; padding-top: 20px")
left_layout.addWidget(self.path_stats_label)

self.controller.solver.solve_completed.connect(self.update_path_stats_label)

self.create_toolbar()

def create_toolbar(self):
    toolbar = QToolBar("Barra de herramientas")
    toolbar.setStyleSheet("background-color: #AAAAAA;")
    self.addToolBar(toolbar)

    maze_action = QAction("Transporte de cajas", self)
    maze_action.triggered.connect(lambda: self.stacked_widget.setCurrentIndex(0))
    toolbar.addAction(maze_action)

    box_order_action = QAction("Reordenamiento de cajas", self)
    box_order_action.triggered.connect(lambda: self.stacked_widget.setCurrentIndex(1))
    toolbar.addAction(box_order_action)

    object_recognition_action = QAction("Identificación de objetos", self)
    object_recognition_action.triggered.connect(lambda: self.stacked_widget.setCurrentIndex(2))
    toolbar.addAction(object_recognition_action)

def update_path_stats_label(self):
    path_stats = self.maze_widget.path_stats
    text = f"""Estadísticas del algoritmo:
    \tNúmero de casillas visitadas: {path_stats["N Visited"]}
    \tTiempo transcurrido: {path_stats["Time Spent"]}
    \tCosto de la ruta: {path_stats["Route Cost"]} pasos"""

    self.path_stats_label.setText(text)

def get_selected_solver(self):
    if self.astar_radio.isChecked():
        return AStarSolver(self.controller.maze)
    elif self.dijkstra_radio.isChecked():
        return BPASolver(self.controller.maze)
    elif self.modified_astar_radio.isChecked():
        return ModifiedAStarSolver(self.controller.maze)

def on_solver_changed(self):
    self.controller.set_solver(self.get_selected_solver())
    self.controller.solver.solve_completed.connect(self.update_path_stats_label)

def solve_maze(self):
    self.controller.solve_maze()

def clear_maze(self):
    self.controller.clear_maze()
    self.maze_widget.clearMaze()

def generate_maze(self):
    self.maze_widget.clearMaze()
    self.controller.generate_maze()
    self.maze_widget.update()

```

10.11. Clase correspondiente a la ventana del reordenamiento de cajas

```

class BoxOrderWindow(QWidget):
    def __init__(self, controller):
        super().__init__()
        self.controller = controller
        self.setWindowTitle("Ordenamiento de cajas")
        self.setMinimumSize(600, 800)
        self.setStyleSheet("background-color: none;")

        title_font = QFont("Arial", 15)
        normal_font = QFont("Arial", 10)

        # Create the main vertical layout
        main_layout = QVBoxLayout()
        self.setLayout(main_layout)

        top_layout = QHBoxLayout()
        buttons_column_layout = QVBoxLayout()
        column1_layout = QVBoxLayout()
        column2_layout = QVBoxLayout()

        column_width = 50 # You can adjust this value to your desired column width

        # Create spacers with the desired width
        spacer_left = QSpacerItem(column_width, 0, QSizePolicy.Fixed, QSizePolicy.Minimum)
        spacer_right = QSpacerItem(column_width, 0, QSizePolicy.Fixed, QSizePolicy.Minimum)

        # Add the spacers and the column layouts to the top layout
        top_layout.addSpacerItem(spacer_left)
        top_layout.addLayout(buttons_column_layout)
        top_layout.addLayout(column1_layout)
        top_layout.addLayout(column2_layout)
        top_layout.addSpacerItem(spacer_right)

        main_layout.addLayout(top_layout)
        top_layout.setAlignment(column1_layout, Qt.AlignLeft)
        top_layout.setAlignment(column2_layout, Qt.AlignRight)

        self.title_label = QLabel("Reordenamiento de cajas")
        self.title_label.setStyleSheet("font-weight: bold;")
        self.title_label.setFont(title_font)
        self.title_label.setContentsMargins(0, 20, 0, 10)

        self.search_button = QPushButton("Buscar solución")
        self.search_button.setFont(normal_font)
        self.search_button.setStyleSheet("background-color: #CCD2CD; margin-left:35px;")
        self.search_button.setFixedSize(250, 40)

        buttons_column_layout.addWidget(self.title_label)
        buttons_column_layout.addWidget(self.search_button)

        # Create the label under the search button (Initially hidden)
        self.search_result_label = QLabel("", self)
        self.search_result_label.setFont(normal_font)
        self.search_result_label.setAlignment(Qt.AlignCenter)
        self.search_result_label.setStyleSheet("color: black; font-size: 20px; padding-top: 20px")
        self.search_result_label.setVisible(True) # Hide initially
        self.search_result_label.setContentsMargins(0, 500, 0, 10)

        buttons_column_layout.addWidget(self.search_result_label)

        buttons_column_layout.addStretch() # Push the button to the top of the column

        # Create and add the global label
        self.global_label = QLabel("Estado inicial", self)
        self.global_label.setAlignment(Qt.AlignLeft)
        self.global_label.setStyleSheet("font-size: 20px; font-weight: bold; padding: 10px; padding-left:
70px;")
        self.global_label.setFixedHeight(40) # Set a fixed height for the label
        column1_layout.addWidget(self.global_label)

        # Spacer to push the rectangles down below the global label
        column1_layout.addStretch()

        self.final_state_label = QLabel("Estado objetivo", self)

```

```

        self.final_state_label.setAlignment(Qt.AlignLeft)
        self.final_state_label.setStyleSheet("font-size: 20px; font-weight: bold; padding: 10px; padding-
right: 420px;")
        self.final_state_label.setFixedHeight(40)
        column2_layout.addWidget(self.final_state_label)

        column2_layout.addStretch()

        # Create a vertical layout for the rectangles
        self.rectangles_init_layout = QVBoxLayout()
        self.rectangles_final_layout = QVBoxLayout()

        column1_layout.addLayout(self.rectangles_init_layout)
        column2_layout.addLayout(self.rectangles_final_layout)

        # Define initial rectangle positions and images
        self.rect_pos_init = {
            "#2A9D8F": (590, 50, 400, 200),
            "#E9C46A": (590, 250, 400, 200),
            "#F4A261": (590, 450, 400, 200),
            "#E76F51": (590, 650, 400, 200),
        }

        self.rect_pos_final = {
            "#2A9D8F": (1180, 50, 400, 200),
            "#E9C46A": (1180, 250, 400, 200),
            "#F4A261": (1180, 450, 400, 200),
            "#E76F51": (1180, 650, 400, 200),
        }

        self.images = [
            r"images\icons\bolt-icon.png",
            r"images\icons\nut.svg",
            r"images\icons\washer.png",
            r"images\icons\nail-icon.png",
        ]

        self.labels = [
            "Tornillo",
            "Tuerca",
            "Arandela",
            "Clavo"
        ]

        self.rectangles_init = [
            {"color": QColor(color), "rect": QRect(*pos), "index": i, "image": QImage(image_path), "label":
label}
            for i, (color, (pos, image_path, label)) in
            enumerate(zip(self.rect_pos_init.keys(), zip(self.rect_pos_init.values(), self.images,
self.labels)))
        ]

        self.rectangles_final = [
            {"color": QColor(color), "rect": QRect(*pos), "index": i, "image": QImage(image_path), "label":
label}
            for i, (color, (pos, image_path, label)) in
            enumerate(zip(self.rect_pos_final.keys(), zip(self.rect_pos_final.values(), self.images,
self.labels)))
        ]

        self.dragging_index = None
        self.dragging_column = None # To track the column being dragged
        self.start_drag_pos = None

        # Add the rectangles to the rectangles layout
        self.add_rectangles_to_layout()

        self.search_button.clicked.connect(self.on_search_clicked)

    def on_search_clicked(self):
        """Call the controller's get_rectangles_order function when the button is clicked."""
        solution = self.controller.search_ppdl_solution(self.rectangles_init, self.rectangles_final)

        # Display solution in a label under the search button
        solution_text = "Solución encontrada : \n"
        #print(solution)

```



```
for i, step in enumerate(solution):
    solution_text += f"\t{i}): {step}\n"
#print(solution_text)
self.search_result_label.setText(solution_text)
self.search_result_label.setVisible(True)

def add_rectangles_to_layout(self):
    for rect_info in self.rectangles_init:
        # Create a widget for each rectangle and add it to the rectangles layout
        rect_widget = QWidget()
        rect_widget.setFixedSize(rect_info["rect"].size())
        self.rectangles_init_layout.addWidget(rect_widget)

    for rect_info in self.rectangles_final:
        rect_widget = QWidget()
        rect_widget.setFixedSize(rect_info["rect"].size())
        self.rectangles_final_layout.addWidget(rect_widget)

def paintEvent(self, event):
    painter = QPainter(self)
    font = QFont("Arial", 12)
    painter.setFont(font)

    for rect_info in self.rectangles_init:
        painter.setBrush(QBrush(rect_info["color"]))
        painter.drawRect(rect_info["rect"])

        # Draw the image inside the rectangle
        image_rect = rect_info["rect"]
        image = rect_info["image"]
        if not image.isNull():
            # Define the size of the image to be a small portion of the rectangle
            image_size = QSize(int(image_rect.width() / 1.5), int(image_rect.height() / 1.5))
            scaled_image = image.scaled(image_size, Qt.KeepAspectRatio, Qt.SmoothTransformation)

            # Calculate the position to center the image within the rectangle
            x_offset = (image_rect.width() - scaled_image.width()) // 2
            y_offset = (image_rect.height() - scaled_image.height()) // 2
            image_position = image_rect.topLeft() + QPoint(x_offset, y_offset)

            # Draw the image in the center of the rectangle
            painter.drawImage(QRect(image_position, scaled_image.size()), scaled_image)

    for rect_info in self.rectangles_final:
        painter.setBrush(QBrush(rect_info["color"]))
        painter.drawRect(rect_info["rect"])

        # Draw the label at the top of the rectangle
        label_rect = rect_info["rect"].adjusted(0, -20, 0, 0) # Adjusted to move label above rectangle
        painter.setPen(QColor("black"))

        # Draw the image inside the rectangle
        image_rect = rect_info["rect"]
        image = rect_info["image"]
        if not image.isNull():
            # Define the size of the image to be a small portion of the rectangle
            image_size = QSize(int(image_rect.width() / 1.5), int(image_rect.height() / 1.5))
            scaled_image = image.scaled(image_size, Qt.KeepAspectRatio, Qt.SmoothTransformation)

            # Calculate the position to center the image within the rectangle
            x_offset = (image_rect.width() - scaled_image.width()) // 2
            y_offset = (image_rect.height() - scaled_image.height()) // 2
            image_position = image_rect.topLeft() + QPoint(x_offset, y_offset)

            # Draw the image in the center of the rectangle
            painter.drawImage(QRect(image_position, scaled_image.size()), scaled_image)

def mousePressEvent(self, event):
    for i, rect_info in enumerate(self.rectangles_init):
        if rect_info["rect"].contains(event.pos()):
            self.dragging_index = i
            self.dragging_column = 1
            self.start_drag_pos = event.pos()
            return

    for i, rect_info in enumerate(self.rectangles_final):
```

```

        if rect_info["rect"].contains(event.pos()):
            self.dragging_index = i
            self.dragging_column = 2
            self.start_drag_pos = event.pos()
            return

    def mouseMoveEvent(self, event):
        if self.dragging_index is not None:
            mouse_y = event.y()
            if self.dragging_column == 1:
                rectangles = self.rectangles_init
            else:
                rectangles = self.rectangles_final

            y_inferior = [rect["rect"].y() + rect["rect"].height() for rect in rectangles]

            #print(self.controller.get_rectangles_order(rectangles))

            # Determine the new position for the dragged rectangle
            new_index = None
            for i, y in enumerate(y_inferior):
                if mouse_y < y:
                    new_index = i
                    break

            # If a new position is found and it's different from the current position
            if new_index is not None and new_index != self.dragging_index:
                # Move the dragged rectangle to the new position
                rect_to_move = rectangles.pop(self.dragging_index)
                rectangles.insert(new_index, rect_to_move)
                # Swap the positions of rectangles to reflect the new order
                if new_index < self.dragging_index:
                    # If the new index is smaller, move rectangles up
                    for i in range(self.dragging_index - 1, new_index - 1, -1):
                        rectangles[i]["rect"], rectangles[i + 1]["rect"] = rectangles[i + 1]["rect"],
rectangles[i]["rect"]
                else:
                    # If the new index is larger, move rectangles down
                    for i in range(self.dragging_index + 1, new_index + 1):
                        rectangles[i]["rect"], rectangles[i - 1]["rect"] = rectangles[i - 1]["rect"],
rectangles[i]["rect"]

                self.dragging_index = new_index

            self.update()

    def mouseReleaseEvent(self, event):
        if self.dragging_index is not None:
            # Reorder rectangles based on their new positions
            if self.dragging_column == 1:
                self.rectangles_init.sort(key=lambda r: r["rect"].top())
            else:
                self.rectangles_final.sort(key=lambda r: r["rect"].top())
            self.dragging_index = None
            self.update()

```

10.12. Clase correspondiente a la ventana de identificación de objetos

```

class ObjectRecognitionWindow(QWidget):
    def __init__(self, controller):
        super().__init__()
        self.setWindowTitle("Etapla 1 - Identificación de objetos")
        self.setMinimumSize(600, 800)
        self.setStyleSheet("background-color: none;")

        self.controller = controller
        self.image_array = None
        self.processed_images = [] # Store processed images as numpy arrays

        # Main horizontal layout
        main_layout = QHBoxLayout()
        main_layout.setAlignment(Qt.AlignTop)

        # Left column layout

```

```
left_column = QVBoxLayout()
left_column.setAlignment(Qt.AlignTop)
left_column.setAlignment(Qt.AlignHCenter)

# Add widgets to the left column
label = QLabel("Identificación de objetos")
label.setStyleSheet("font-size: 18pt; padding: 10px; font-weight: bold;")
left_column.addWidget(label)

# Button to load image
self.load_button = QPushButton("Cargar imagen")
self.load_button.setStyleSheet("font-size: 12pt; padding: 5px; background-color: rgb(200, 200, 200);")
self.load_button.clicked.connect(self.load_image) # Connect to function
left_column.addWidget(self.load_button)

# Image display label
self.image_label = QLabel("No se ha cargado una imagen")
self.image_label.setAlignment(Qt.AlignCenter)
self.image_label.setStyleSheet("border: 1px solid black; padding: 5px;")
left_column.addWidget(self.image_label)

# Recognition algorithm label
self.algorithm_label = QLabel("Algoritmo de reconocimiento:")
self.algorithm_label.setAlignment(Qt.AlignLeft)
self.algorithm_label.setStyleSheet("padding: 5px; font-size: 12pt;")
left_column.addWidget(self.algorithm_label)

# Horizontal layout for radio buttons
algo_layout = QHBoxLayout()
algo_layout.setAlignment(Qt.AlignHCenter)

# KMeans radio button
self.kmeans_radio = QRadioButton("K-Means")
self.kmeans_radio.setStyleSheet("padding-right: 30%; font-size: 10pt;")
self.kmeans_radio.setChecked(True) # Default selection
algo_layout.addWidget(self.kmeans_radio)

# KNN radio button
self.knn_radio = QRadioButton("KNN")
self.knn_radio.setStyleSheet("padding-left: 30%; font-size: 10pt;")
algo_layout.addWidget(self.knn_radio)

# Group radio buttons
self.algo_group = QButtonGroup(self)
self.algo_group.addButton(self.kmeans_radio)
self.algo_group.addButton(self.knn_radio)

left_column.addLayout(algo_layout)

self.launch = QPushButton("Ejecutar algoritmo")
self.launch.setStyleSheet("font-size: 12pt; padding: 5px; margin: 10px 50px; background-color: rgb(200, 200, 200);")
self.launch.clicked.connect(self.run_algorithm)
left_column.addWidget(self.launch)

# **Vote Result Label**
self.vote_result_label = QLabel("")
self.vote_result_label.setAlignment(Qt.AlignCenter)
self.vote_result_label.setStyleSheet("font-size: 14pt; font-weight: bold; color: blue; padding-top: 10px;")
left_column.addWidget(self.vote_result_label)

# **Confidence Label**
self.confidence_label = QLabel("")
self.confidence_label.setAlignment(Qt.AlignCenter)
self.confidence_label.setStyleSheet("font-size: 14pt; font-weight: bold; color: green; padding-bottom: 10px;")
left_column.addWidget(self.confidence_label)

main_layout.addLayout(left_column, 1)

### RIGHT COLUMN (PROCESSED IMAGES) ###
# Right column layout
right_column = QVBoxLayout()
right_column.setAlignment(Qt.AlignTop)
```

```
# Label for preprocessed image
self.preprocessed_image_text = QLabel("Imagen procesada junto a la envolvente convexa")
self.preprocessed_image_text.setVisible(False)
self.preprocessed_image_text.setAlignment(Qt.AlignCenter)
self.preprocessed_image_text.setStyleSheet("padding-top:30px; font-size: 14pt")

self.processed_image_label = QLabel("")
self.processed_image_label.setAlignment(Qt.AlignCenter)
self.processed_image_label.setStyleSheet("padding:5px 50px")

right_column.addWidget(self.preprocessed_image_text)
right_column.addWidget(self.processed_image_label)

# Label for features title
self.features_label = QLabel("Características extraídas")
self.features_label.setAlignment(Qt.AlignHCenter)
self.features_label.setStyleSheet("font-weight: bold; font-size:12pt; padding:0px;")
self.features_label.setVisible(False)
right_column.addWidget(self.features_label)

# Grid layout for displaying features (3 rows x 3 columns)
self.features_grid = QGridLayout()
self.features_grid.setAlignment(Qt.AlignHCenter)
# Add layout for displaying preprocessed features
self.features_container = QWidget()
self.features_container.setStyleSheet("padding: 0px;")
self.features_container.setLayout(self.features_grid)
right_column.addWidget(self.features_container)

main_layout.addLayout(right_column, 3) # Right column takes more space

self.setLayout(main_layout)

def load_image(self):
    """ Open file dialog to select an image and display it. """
    self.reset_labels()

    options = QFileDialog.Options()
    file_path, _ = QFileDialog.getOpenFileName(
        self, "Open Image File", "", "Images (*.png *.jpg *.jpeg *.bmp *.gif)", options=options
    )
    if file_path:
        # Load image with PIL
        pil_image = Image.open(file_path)
        # Convert to NumPy array and store it
        self.image_array = np.array(pil_image)

        # Display the image in the QLabel
        pixmap = QPixmap(file_path)
        self.image_label.setPixmap(pixmap.scaled(400, 400, Qt.KeepAspectRatio))
        self.image_label.setText("") # Clear placeholder text

def run_algorithm(self):
    """ Send the image array to the controller when launch button is clicked. """
    if self.image_array is None:
        self.image_label.setText("Por favor, seleccione una imagen antes de ejecutar un algoritmo.")
        return

    # Determine selected algorithm
    algorithm = "K-Means" if self.kmeans_radio.isChecked() else "KNN"

    # Call controller function with image and selected algorithm
    result_list = self.controller.launch_algorithm(algorithm, self.image_array)

    if len(result_list) == 5:
        # KNN results
        vote_result, confidence, processed_image, features, scaled_features = result_list

        # **Update Labels**
        self.vote_result_label.setText(f"Clase resultante: {vote_result}")
        self.confidence_label.setText(f"Confianza: {confidence * 100:.2f}%")
    else:
        vote_group, vote_result, confidence, processed_image, features, scaled_features = result_list
```

```

        self.vote_result_label.setText(f"Grupo resultante: {vote_group} - Clase: {vote_result}")
        self.confidence_label.setText(f"Precisión del agrupamiento: {confidence * 100:.2f}%")

    # Display Preprocessed Images
    self.update_preprocessed_image(processed_image)
    self.update_preprocessed_features(features, scaled_features)

def update_preprocessed_image(self, image_array):
    """
    Update the QLabel with the new preprocessed image.

    Parameters:
        image_array (np.array): The processed image to display.
    """
    if image_array is not None:
        # Convert NumPy array to QImage
        height, width, channels = image_array.shape
        bytes_per_line = channels * width
        q_image = QImage(image_array.data, width, height, bytes_per_line, QImage.Format_RGB888)

        # Convert QImage to QPixmap and update QLabel with smaller size
        pixmap = QPixmap.fromImage(q_image)
        self.preprocessed_image_text.setVisible(True)
        self.processed_image_label.setPixmap(pixmap.scaled(500, 500, Qt.KeepAspectRatio)) # Scale down
to 200x200
        self.processed_image_label.setVisible(True)

def update_preprocessed_features(self, features, scaled_features):
    """
    Update the grid of preprocessed features in the right column.

    Parameters:
        features (dict or list): Preprocessed features to display.
    """
    # Assuming features is a dictionary with keys and values
    row = 0
    col = 0
    self.features_label.setVisible(True)
    if isinstance(features, dict):
        for feature_name, feature_value in features.items():
            # Add feature name as QLabel
            feature_name_label = QLabel(f"{feature_name}: {feature_value:.2f}")
            feature_name_label.setStyleSheet("padding: 5px 20px; font-size: 12pt;")
            self.features_grid.addWidget(feature_name_label, row, col)

            row += 1

    row = 0
    col = 1
    if isinstance(features, dict):
        for feature_name, feature_value in scaled_features.items():
            # Add feature name as QLabel
            feature_name_label = QLabel(f"{feature_name}: {feature_value:.2f}")
            feature_name_label.setStyleSheet("padding: 5px 20px; font-size: 12pt;")
            self.features_grid.addWidget(feature_name_label, row, col)

            row += 1

def reset_labels(self):
    for i in reversed(range(self.features_grid.count())):
        widget = self.features_grid.itemAt(i).widget()
        if widget is not None:
            widget.setVisible(False)
            widget.deleteLater()

    self.vote_result_label.setText("")
    self.confidence_label.setText("")
    self.features_label.setVisible(False)
    self.preprocessed_image_text.setVisible(False)
    self.processed_image_label.setVisible(False)

```